

Training Dynamics of Overparameterized Neural Networks

Shreyas Kalvankar

MSc Computer Science, TU Delft

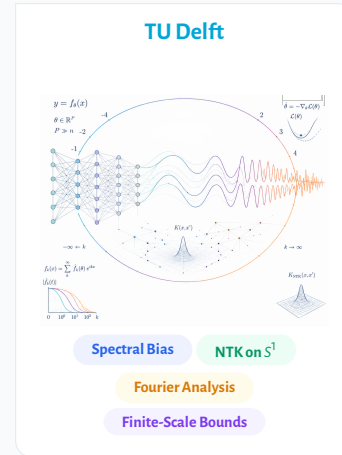
Advisor: Dr. D. M. J. Tax, Pattern Recognition & Bioinformatics Group

Supervisor: Dr. A. Heinlein, Numerical Analysis Group

Committee Member: Prof. Dr. A. Papapantoleon, Applied Probability Group

May 2026

TU Delft



2026-06-11

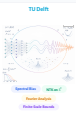
Training Dynamics of Overparameterized Neural Networks

Shreyas Kalvankar
MSc Computer Science, TU Delft

Advisor: Dr. D. M. J. Tax, Pattern Recognition & Bioinformatics Group
Supervisor: Dr. A. Heinlein, Numerical Analysis Group
Committee Member: Prof. Dr. A. Papapantoleon, Applied Probability Group

May 2026

TU Delft



Supervised Learning: Fitting data to a function



2026-06-11

- Network = a rule turning input into output, i.e. a function.
- Given example (input, output) pairs; want a function that agrees with them and *generalizes* (sensible answers on unseen inputs).
- How: adjust the function until outputs match; the squared error measures how far off.
- Can't try arbitrary functions one by one → fix a parameterized family; learning = tuning the parameters.
- Keep “unknowns” in mind: next slide counts parameters vs data points like unknowns vs equations.
- Next: what is such a function built from?



Supervised Learning: Fitting data to a function

Data: n observations $(x_i, f^*(x_i))$, examples of an unknown target function f^* .



2026-06-11

- Network = a rule turning input into output, i.e. a function.
- Given example (input, output) pairs; want a function that agrees with them and *generalizes* (sensible answers on unseen inputs).
- How: adjust the function until outputs match; the squared error measures how far off.
- Can't try arbitrary functions one by one → fix a parameterized family; learning = tuning the parameters.
- Keep “unknowns” in mind: next slide counts parameters vs data points like unknowns vs equations.
- Next: what is such a function built from?



Supervised Learning: Fitting data to a function

Data: n observations $(x_i, f^*(x_i))$, examples of an unknown target function f^* .

Model: a learned prediction function $f(x)$.



2026-06-11

- Network = a rule turning input into output, i.e. a function.
- Given example (input, output) pairs; want a function that agrees with them and *generalizes* (sensible answers on unseen inputs).
- How: adjust the function until outputs match; the squared error measures how far off.
- Can't try arbitrary functions one by one → fix a parameterized family; learning = tuning the parameters.
- Keep “unknowns” in mind: next slide counts parameters vs data points like unknowns vs equations.
- Next: what is such a function built from?

Data: n observations $(x_i, f^*(x_i))$, examples of an unknown target function f^* .

Model: a learned prediction function $f(x)$.

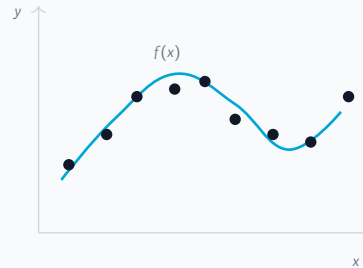


Supervised Learning: Fitting data to a function

Data: n observations $(x_i, f^*(x_i))$, examples of an unknown target function f^* .

Model: a learned prediction function $f(x)$.

Aim: predict the target output $f^*(x)$ for a new, unseen input x .



2026-06-11

Data: a collection $\{(x_i, f^*(x_i))\}$, examples of an unknown target function f^* .
Model: a learned prediction function $f(x)$.
Aim: predict the target output $f^*(x)$ for a new, unseen input x .



- Network = a rule turning input into output, i.e. a function.
- Given example (input, output) pairs; want a function that agrees with them and *generalizes* (sensible answers on unseen inputs).
- How: adjust the function until outputs match; the squared error measures how far off.
- Can't try arbitrary functions one by one → fix a parameterized family; learning = tuning the parameters.
- Keep “unknowns” in mind: next slide counts parameters vs data points like unknowns vs equations.
- Next: what is such a function built from?

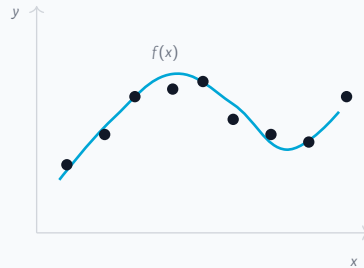
Supervised Learning: Fitting data to a function

Data: n observations $(x_i, f^*(x_i))$, examples of an unknown target function f^* .

Model: a learned prediction function $f(x)$.

Aim: predict the target output $f^*(x)$ for a new, unseen input x .

How: choose f so its predictions are close to the target function on the observed data, i.e. minimise $\mathcal{L}(f) = \frac{1}{n} \sum_{i=1}^n \underbrace{(f(x_i) - f^*(x_i))}_{:= r_i}^2$.



2026-06-11

- Network = a rule turning input into output, i.e. a function.
- Given example (input, output) pairs; want a function that agrees with them and *generalizes* (sensible answers on unseen inputs).
- How: adjust the function until outputs match; the squared error measures how far off.
- Can't try arbitrary functions one by one → fix a parameterized family; learning = tuning the parameters.
- Keep “unknowns” in mind: next slide counts parameters vs data points like unknowns vs equations.
- Next: what is such a function built from?

Data: a observations $(x_i, f^*(x_i))$, examples of an unknown target function f^* .
Model: a learned prediction function $f(x)$.
Aim: predict the target output $f^*(x)$ for a new, unseen input x .
How: choose f so its predictions are close to the target function on the observed data, i.e. minimise $\mathcal{L}(f) = \frac{1}{n} \sum_{i=1}^n (f(x_i) - f^*(x_i))^2$.



Supervised Learning: Fitting data to a function

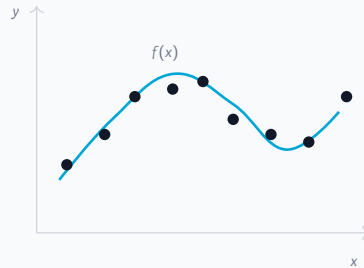
Data: n observations $(x_i, f^*(x_i))$, examples of an unknown target function f^* .

Model: a learned prediction function $f(x)$.

Aim: predict the target output $f^*(x)$ for a new, unseen input x .

How: choose f so its predictions are close to the target function on the observed data, i.e. minimise $\mathcal{L}(f) = \frac{1}{n} \sum_{i=1}^n \underbrace{(f(x_i) - f^*(x_i))}_{:= r_i}^2$.

In practice: we cannot search over all possible functions, so we fix a parameterized family $f_\theta(x)$ and learn by adjusting the parameters θ . The parameters are the unknowns the data must pin down.



2026-06-11

Data: a observations $(x_i, f^*(x_i))$, examples of an unknown target function f^* .
Model: a learned prediction function $f(x)$.
Aim: predict the target output $f^*(x)$ for a new, unseen input x .
How: choose f so its predictions are close to the target function on the observed data, i.e. minimise $\mathcal{L}(f) = \frac{1}{n} \sum_{i=1}^n (f(x_i) - f^*(x_i))^2$.
In practice: we cannot search over all possible functions, so we fix a parameterized family $f_\theta(x)$ and learn by adjusting the parameters θ . The parameters are the unknowns the data must pin down.



- Network = a rule turning input into output, i.e. a function.
- Given example (input, output) pairs; want a function that agrees with them and *generalizes* (sensible answers on unseen inputs).
- How: adjust the function until outputs match; the squared error measures how far off.
- Can't try arbitrary functions one by one → fix a parameterized family; learning = tuning the parameters.
- Keep “unknowns” in mind: next slide counts parameters vs data points like unknowns vs equations.
- Next: what is such a function built from?

Overparameterization: many ways to fit the same data

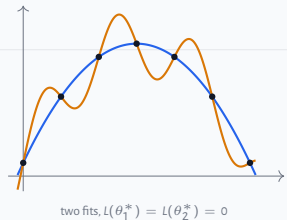
When many solutions fit the data, the learning process itself becomes important.

A linear analogy

3 equations, 3 unknowns $\Rightarrow$ one solution

3 equations, 4 unknowns $\Rightarrow$ infinitely many
independent equations, consistent system

One spare unknown, and the data no longer pins down the answer:
a whole line of solutions fits.



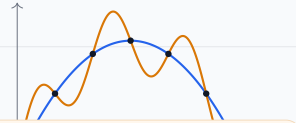
2026-06-11

- 3 eq / 3 unknowns: one solution; 3 eq / 4 unknowns: a line of solutions.
- Network: each data point = one constraint on θ .
- “Solution” = near-zero training error.
- Example: CIFAR-10 has 50k images; WRN-28-10 has many more parameters.
- Large networks can generalize, but also fit random labels.
- Key question: which fit does gradient descent pick?
- Teaser: coarse before fine.
- Inset: seven shared points, two zero-error interpolants. The data does not distinguish them; the question is which one training finds.



Overparameterization: many ways to fit the same data

When many solutions fit the data, the learning process itself becomes important.



A linear analogy

3 equations, 3 unknowns $\Rightarrow$ one solution

3 equations, 4 unknowns $\Rightarrow$ infinitely many
independent equations, consistent system

One spare unknown, and the data no longer pins down the answer:
a whole line of solutions fits.

The same situation in a neural network

Each training point is one constraint on the parameters θ :

$$f_{\theta}(x_i) \approx y_i, \text{ for } i = 1, \dots, n.$$

A solution: a parameter setting with near-zero training error.

CIFAR-10 has **50k** training images (Krizhevsky, 2009);
WRN-28-10 has **36.5M** parameters and about **4%** test error (Zagoruyko & Komodakis, 2016).

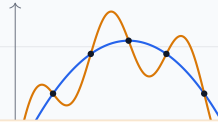
2026-06-11



- 3 eq / 3 unknowns: one solution; 3 eq / 4 unknowns: a line of solutions.
- Network: each data point = one constraint on θ .
- “Solution” = near-zero training error.
- Example: CIFAR-10 has 50k images; WRN-28-10 has many more parameters.
- Large networks can generalize, but also fit random labels.
- Key question: which fit does gradient descent pick?
- Teaser: coarse before fine.
- Inset: seven shared points, two zero-error interpolants. The data does not distinguish them; the question is which one training finds.

Overparameterization: many ways to fit the same data

When many solutions fit the data, the learning process itself becomes important.



A linear analogy

3 equations, 3 unknowns $\Rightarrow$ one solution

3 equations, 4 unknowns $\Rightarrow$ infinitely many
independent equations, consistent system

One spare unknown, and the data no longer pins down the answer:
a whole line of solutions fits.

The same situation in a neural network

Each training point is one constraint on the parameters θ :

$$f_{\theta}(x_i) \approx y_i, \text{ for } i = 1, \dots, n.$$

A solution: a parameter setting with near-zero training error.

CIFAR-10 has **50k** training images (Krizhevsky, 2009);
WRN-28-10 has **36.5M** parameters and about **4%** test error (Zagoruyko & Komodakis, 2016).

Large networks can also fit random labels, so fitting the training set alone does not explain generalization (Zhang et al., 2017).

Empirical observation: models often learn simple structure before complex detail (Rahaman et al., 2019; Boursier & Flammarion, 2024).

2026-06-11

- 3 eq / 3 unknowns: one solution; 3 eq / 4 unknowns: a line of solutions.
- Network: each data point = one constraint on θ .
- “Solution” = near-zero training error.
- Example: CIFAR-10 has 50k images; WRN-28-10 has many more parameters.
- Large networks can generalize, but also fit random labels.
- Key question: which fit does gradient descent pick?
- Teaser: coarse before fine.
- Inset: seven shared points, two zero-error interpolants. The data does not distinguish them; the question is which one training finds.



Why study training behaviour?

The broader goal is to understand learning, not just report a final test error.

- Once many fits are possible, improving a model is not only about making it larger; it is also about understanding what training is actually doing.

2026-06-11

- Keep this high level.
- Not claiming this solves energy or deployment.
- Point: training behaviour matters because training is costly and opaque.
- Compute and data-centre energy are growing quickly.
- Do not explain thesis yet; next slides show the mechanism.
- Transition: what does training learn first?

Why study training behaviour?

The broader goal is to understand learning, not just report a final test error.

- ▶ Once many fits are possible, improving a model is not only about making it larger; it is also about understanding what training is actually doing.
- ▶ This matters more as modern AI systems become expensive and energy-intensive to train: the compute used to train large-scale AI models is reported to double roughly every five months (Stanford AI Index, 2025); the Llama 3.1 family alone used 39.3M H100 GPU-hours for training (Meta AI, 2024).

2026-06-11

- Keep this high level.
- Not claiming this solves energy or deployment.
- Point: training behaviour matters because training is costly and opaque.
- Compute and data-centre energy are growing quickly.
- Do not explain thesis yet; next slides show the mechanism.
- Transition: what does training learn first?

Why study training behaviour?

The broader goal is to understand learning, not just report a final test error.

- ▶ Once many fits are possible, improving a model is not only about making it larger; it is also about understanding what training is actually doing.
- ▶ This matters more as modern AI systems become expensive and energy-intensive to train: the compute used to train large-scale AI models is reported to double roughly every five months (Stanford AI Index, 2025); the Llama 3.1 family alone used 39.3M H100 GPU-hours for training (Meta AI, 2024).

Why study training behaviour?

The broader goal is to understand learning, not just report a final test error.

- ▶ Once many fits are possible, improving a model is not only about making it larger; it is also about understanding what training is actually doing.
- ▶ This matters more as modern AI systems become expensive and energy-intensive to train: the compute used to train large-scale AI models is reported to double roughly every five months (Stanford AI Index, 2025); the Llama 3.1 family alone used 39.3M H100 GPU-hours for training (Meta AI, 2024).
- ▶ So before asking for a bigger model, we can ask: **what does gradient descent learn first?**

2026-06-11

- Keep this high level.
- Not claiming this solves energy or deployment.
- Point: training behaviour matters because training is costly and opaque.
- Compute and data-centre energy are growing quickly.
- Do not explain thesis yet; next slides show the mechanism.
- Transition: what does training learn first?

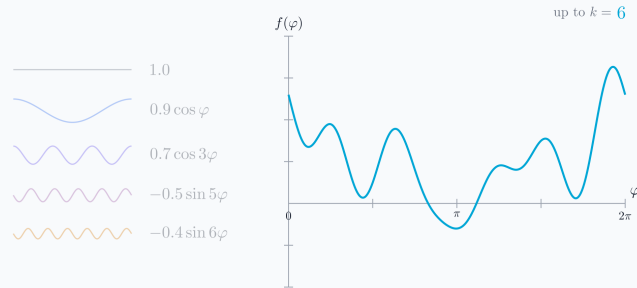
Why study training behaviour?

The broader goal is to understand learning, not just report a final test error.

- ▶ Once many fits are possible, improving a model is not only about making it larger; it is also about understanding what training is actually doing.
- ▶ This matters more as modern AI systems become expensive and energy-intensive to train: the compute used to train large-scale AI models is reported to double roughly every five months (Stanford AI Index, 2025); the Llama 3.1 family alone used 39.3M H100 GPU-hours for training (Meta AI, 2024).
- ▶ So before asking for a bigger model, we can ask: **what does gradient descent learn first?**

A concrete target

To watch that choice happen, take a target built from a coarse shape plus finer and finer detail.



Frequency components

- ▶ Each piece (i.e. a *mode*) is one frequency, from slow waves to fast ones.
- ▶ Low frequencies set the coarse shape; high frequencies add the fine detail.
- ▶ The target is a short sum of these pieces.

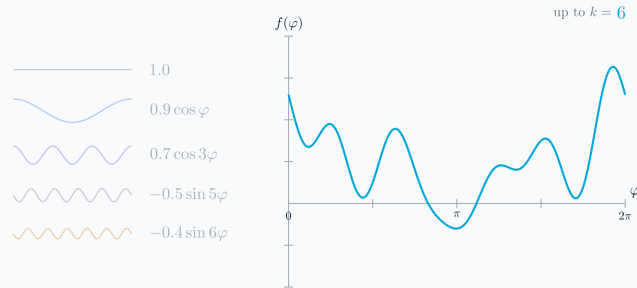
2026-06-11

- Target = short sum of frequencies, coarse $\rightarrow$ fine.
- The object the rest of the talk tracks.
- Sets up the demo: what appears first?



A concrete target

To watch that choice happen, take a target built from a coarse shape plus finer and finer detail.



Frequency components

- ▶ Each piece (i.e. a *mode*) is one frequency, from slow waves to fast ones.
- ▶ Low frequencies set the coarse shape; high frequencies add the fine detail.
- ▶ The target is a short sum of these pieces.

What gets learned first?

2026-06-11

To watch that choice happen, take a target built from a coarse shape plus finer and finer detail.



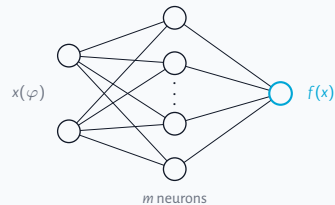
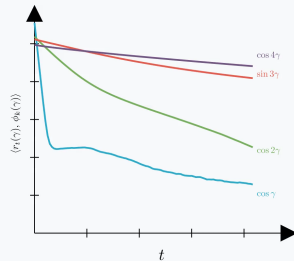
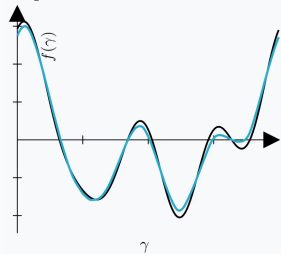
What gets learned first?

- Target = short sum of frequencies, coarse $\rightarrow$ fine.
- The object the rest of the talk tracks.
- Sets up the demo: what appears first?

Watch one network learn it

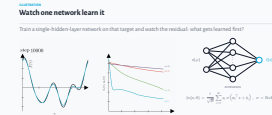
Train a single-hidden-layer network on that target and watch the residual: what gets learned first?

step 10000



$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

2026-06-11

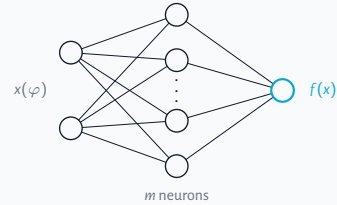
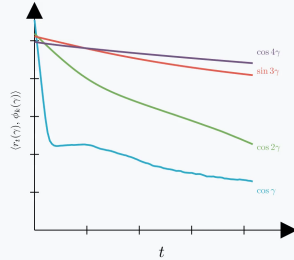
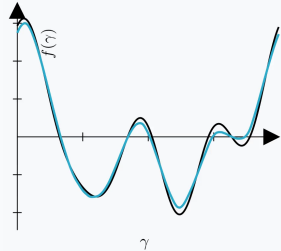


- Train the single-hidden-layer network (the model the whole thesis analyses) on the opener's target.
- Three panels: fit climbing to the target; each mode's residual decaying during training; output rebuilt as a partial sum, coarse to fine.
- Takeaway: low-frequency structure learned before high-frequency detail.

Watch one network learn it

Train a single-hidden-layer network on that target and watch the residual: what gets learned first?

step 10000

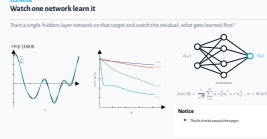


$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

Notice

- ▶ The fit climbs toward the target.

2026-06-11

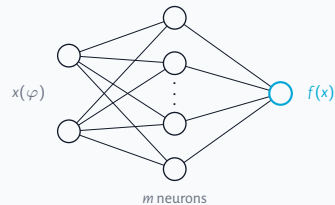
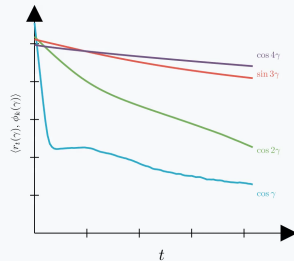
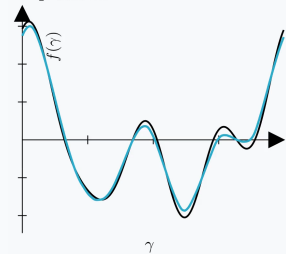


- Train the single-hidden-layer network (the model the whole thesis analyses) on the opener's target.
- Three panels: fit climbing to the target; each mode's residual decaying during training; output rebuilt as a partial sum, coarse to fine.
- Takeaway: low-frequency structure learned before high-frequency detail.

Watch one network learn it

Train a single-hidden-layer network on that target and watch the residual: what gets learned first?

step 10000



$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

Notice

- ▶ The fit climbs toward the target.
- ▶ Low-frequency modes die first.

2026-06-11

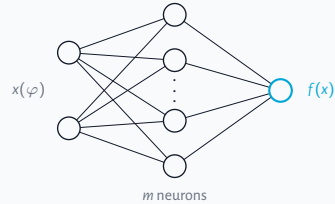
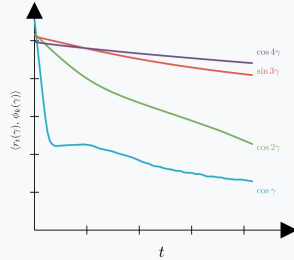
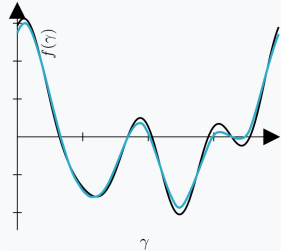


- Train the single-hidden-layer network (the model the whole thesis analyses) on the opener's target.
- Three panels: fit climbing to the target; each mode's residual decaying during training; output rebuilt as a partial sum, coarse to fine.
- Takeaway: low-frequency structure learned before high-frequency detail.

Watch one network learn it

Train a single-hidden-layer network on that target and watch the residual: what gets learned first?

step 10000

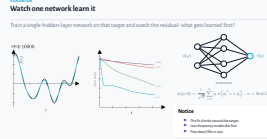


$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

Notice

- ▶ The fit climbs toward the target.
- ▶ Low-frequency modes die first.
- ▶ Fine detail fills in last.

2026-06-11

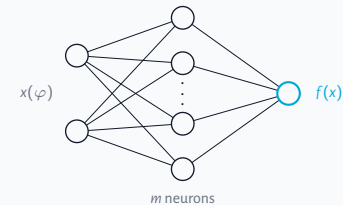
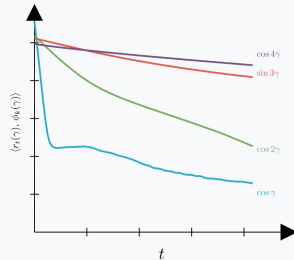
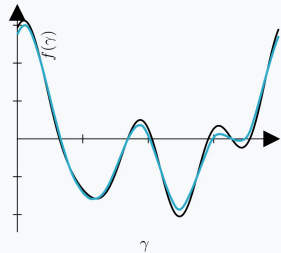


- Train the single-hidden-layer network (the model the whole thesis analyses) on the opener's target.
- Three panels: fit climbing to the target; each mode's residual decaying during training; output rebuilt as a partial sum, coarse to fine.
- Takeaway: low-frequency structure learned before high-frequency detail.

Watch one network learn it

Train a single-hidden-layer network on that target and watch the residual: what gets learned first?

step 10000



$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

Notice

- ▶ The fit climbs toward the target.
- ▶ Low-frequency modes die first.
- ▶ Fine detail fills in last.

The coarse shape appears first; the fine detail comes last.

2026-06-11



- Train the single-hidden-layer network (the model the whole thesis analyses) on the opener's target.
- Three panels: fit climbing to the target; each mode's residual decaying during training; output rebuilt as a partial sum, coarse to fine.
- Takeaway: low-frequency structure learned before high-frequency detail.

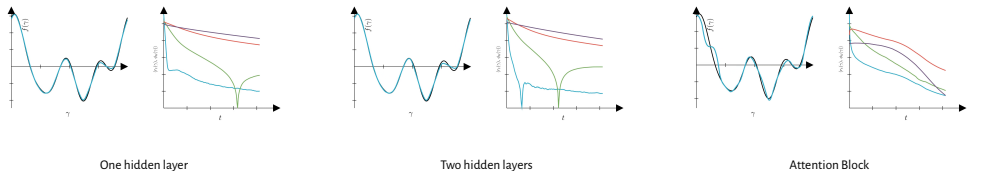
Why does gradient descent learn simple structure first?

This pattern has a name: the frequency principle, also called spectral bias (Xu et al., 2019).

Thesis in one sentence

Derive the exact learning speed of every frequency in an idealized network, then quantify how far that exact picture persists in a realistic one: finitely many training points, finitely many neurons.

Frequency ordering in different networks.



2026-06-11

- Spectral bias: GD-trained networks learn low-frequency structure before high-frequency detail.
- I.e. first the smooth, global shape, only later the oscillatory parts.
- These examples show it: different architectures, same pattern — low modes early, high modes delayed.
- Thesis asks: can we explain this from the training dynamics?
- Main idea: recast spectral bias as an operator eigenvalue problem; the operator's eigenvalues set which frequencies learn fast vs slow.
- Three guiding questions: what controls mode-wise rates? does it survive finite data? does it survive finite width?



Why does gradient descent learn simple structure first?

This pattern has a name: the frequency principle, also called spectral bias (Xu et al., 2019).

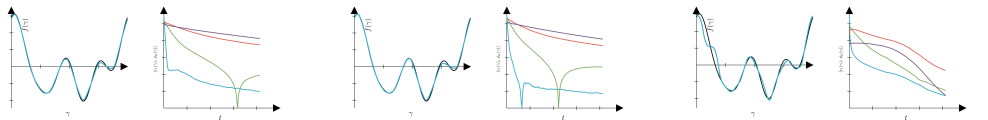
Thesis in one sentence

Derive the exact learning speed of every frequency in an idealized network, then quantify how far that exact picture persists in a realistic one: finitely many training points, finitely many neurons.

Three questions

- 1 What sets the learning speed of each frequency?

Frequency ordering in different networks.



One hidden layer

Two hidden layers

Attention Block

2026-06-11

- Spectral bias: GD-trained networks learn low-frequency structure before high-frequency detail.
- I.e. first the smooth, global shape, only later the oscillatory parts.
- These examples show it: different architectures, same pattern — low modes early, high modes delayed.
- Thesis asks: can we explain this from the training dynamics?
- Main idea: recast spectral bias as an operator eigenvalue problem; the operator's eigenvalues set which frequencies learn fast vs slow.
- Three guiding questions: what controls mode-wise rates? does it survive finite data? does it survive finite width?



Why does gradient descent learn simple structure first?

This pattern has a name: the frequency principle, also called spectral bias (Xu et al., 2019).

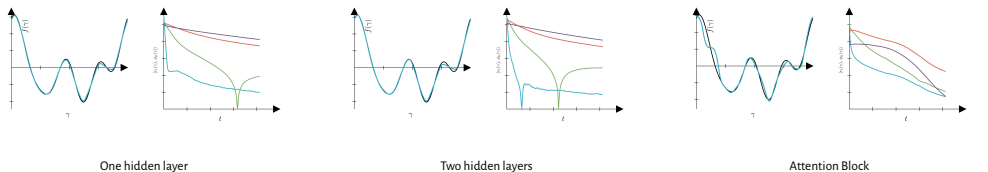
Thesis in one sentence

Derive the exact learning speed of every frequency in an idealized network, then quantify how far that exact picture persists in a realistic one: finitely many training points, finitely many neurons.

Three questions

- 1 What sets the learning speed of each frequency?
- 2 Does the answer survive when the network sees only n sampled points? (RQ1)

Frequency ordering in different networks.



2026-06-11



- Spectral bias: GD-trained networks learn low-frequency structure before high-frequency detail.
- I.e. first the smooth, global shape, only later the oscillatory parts.
- These examples show it: different architectures, same pattern — low modes early, high modes delayed.
- Thesis asks: can we explain this from the training dynamics?
- Main idea: recast spectral bias as an operator eigenvalue problem; the operator's eigenvalues set which frequencies learn fast vs slow.
- Three guiding questions: what controls mode-wise rates? does it survive finite data? does it survive finite width?

Why does gradient descent learn simple structure first?

This pattern has a name: the frequency principle, also called spectral bias (Xu et al., 2019).

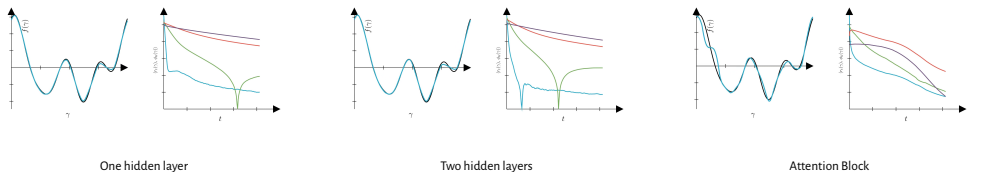
Thesis in one sentence

Derive the exact learning speed of every frequency in an idealized network, then quantify how far that exact picture persists in a realistic one: finitely many training points, finitely many neurons.

Three questions

- 1 What sets the learning speed of each frequency?
- 2 Does the answer survive when the network sees only n sampled points? (RQ1)
- 3 Does it survive with finitely many neurons, at the start of training and during it? (RQ2)

Frequency ordering in different networks.



2026-06-11

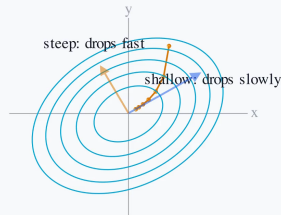


- Spectral bias: GD-trained networks learn low-frequency structure before high-frequency detail.
- I.e. first the smooth, global shape, only later the oscillatory parts.
- These examples show it: different architectures, same pattern — low modes early, high modes delayed.
- Thesis asks: can we explain this from the training dynamics?
- Main idea: recast spectral bias as an operator eigenvalue problem; the operator's eigenvalues set which frequencies learn fast vs slow.
- Three guiding questions: what controls mode-wise rates? does it survive finite data? does it survive finite width?

The error is a landscape

Seen from above, the contour rings are the bowl's height; training rolls toward the bottom.

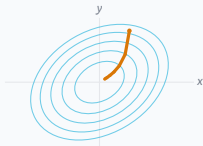
recall the loss: $\mathcal{L}(f) = \frac{1}{n} \sum_i (f(x_i) - f^*(x_i))^2$



- Picture the error as a landscape: high where wrong, low where it fits.
- Training rolls downhill, steepest way down — hence gradient descent.
- Steps shrink near the bottom: smaller error $\rightarrow$ smaller correction (slope $\propto$ error left).
- Same bowl from above: rings are contours (equal height); close rings = steep, wide = shallow.
- This bowl has a steep direction and a shallow one — the distinction the next slide builds on.

Each direction has its own rate

Training makes the error small by walking downhill, along the steepest drop.



In x, y axes the two directions are coupled.

$$\text{recall the loss: } \mathcal{L}(f) = \frac{1}{n} \sum_i (f(x_i) - f^*(x_i))^2$$

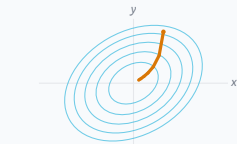
2026-06-11



- What training does: make the error small, by walking downhill (negative gradient) — gradient descent.
- Question this slide answers: why do different directions learn at different speeds?
- Picture a stretched bowl. In its own tilted coordinates the descent path mixes the two directions → no clean rate for either.
- Fix: use the bowl's own axes. These special directions (eigenvectors) are the ones the dynamics never mixes.
- Along one, the error falls in proportion to what's left (the ODE) — same law as cooling coffee / radioactive half-life → exponential decay, $\alpha(t) = \alpha(0)e^{-\lambda t}$.
- Rate λ = eigenvalue: large is fast, small is slow.
- Everything after works out these directions and rates for a real network on the circle.

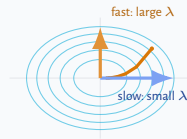
Each direction has its own rate

Training makes the error small by walking downhill, along the steepest drop.



In x, y axes the two directions are coupled.

⇒
the bowl's own
axes



Along each axis the error falls on its own.

$$\text{recall the loss: } \mathcal{L}(f) = \frac{1}{n} \sum_i (f(x_i) - f^*(x_i))^2$$

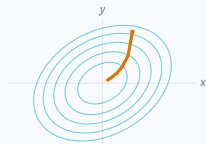
2026-06-11



- What training does: make the error small, by walking downhill (negative gradient) — gradient descent.
- Question this slide answers: why do different directions learn at different speeds?
- Picture a stretched bowl. In its own tilted coordinates the descent path mixes the two directions → no clean rate for either.
- Fix: use the bowl's own axes. These special directions (eigenvectors) are the ones the dynamics never mixes.
- Along one, the error falls in proportion to what's left (the ODE) — same law as cooling coffee / radioactive half-life → exponential decay, $\alpha(t) = \alpha(0)e^{-\lambda t}$.
- Rate λ = eigenvalue: large is fast, small is slow.
- Everything after works out these directions and rates for a real network on the circle.

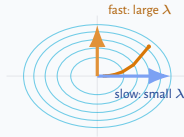
Each direction has its own rate

Training makes the error small by walking downhill, along the steepest drop.



In x, y axes the two directions are coupled.

⇒
the bowl's own
axes



Along each axis the error falls on its own.

$$\text{recall the loss: } \mathcal{L}(f) = \frac{1}{n} \sum_i (f(x_i) - f^*(x_i))^2$$

Along one direction, the error falls in proportion to how much is left:

$$\frac{d}{dt} \alpha(t) = -\lambda \alpha(t)$$

2026-06-11

Training makes the error small by walking downhill, along the steepest drop.



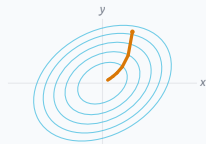
Along one direction, the error falls in proportion to how much is left.

$$\frac{d}{dt} \alpha(t) = -\lambda \alpha(t)$$

- What training does: make the error small, by walking downhill (negative gradient) — gradient descent.
- Question this slide answers: why do different directions learn at different speeds?
- Picture a stretched bowl. In its own tilted coordinates the descent path mixes the two directions → no clean rate for either.
- Fix: use the bowl's own axes. These special directions (eigenvectors) are the ones the dynamics never mixes.
- Along one, the error falls in proportion to what's left (the ODE) — same law as cooling coffee / radioactive half-life → exponential decay, $\alpha(t) = \alpha(0)e^{-\lambda t}$.
- Rate λ = eigenvalue: large is fast, small is slow.
- Everything after works out these directions and rates for a real network on the circle.

Each direction has its own rate

Training makes the error small by walking downhill, along the steepest drop.

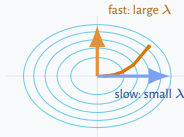


In x, y axes the two directions are coupled.

Along one direction, the error falls in proportion to how much is left:

$$\frac{d}{dt} \alpha(t) = -\lambda \alpha(t)$$

⇒
the bowl's own
axes

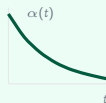


Along each axis the error falls on its own.

so it decays exponentially:

$$\alpha(t) = \alpha(0) e^{-\lambda t}$$

like cooling or half-life; λ is the rate.



recall the loss: $\mathcal{L}(f) = \frac{1}{n} \sum_i (f(x_i) - f^*(x_i))^2$

2026-06-11

statement
Each direction has its own rate

Training makes the error small by walking downhill, along the steepest drop.



In x, y axes the two directions are coupled.

Along one direction, the error falls in proportion to how much is left:

$$\frac{d}{dt} \alpha(t) = -\lambda \alpha(t)$$

In its own axes the two directions are uncoupled.

Along each axis the error falls on its own.

so it decays exponentially:

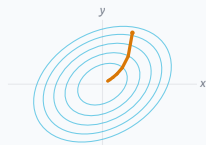
$$\alpha(t) = \alpha(0) e^{-\lambda t}$$

like cooling or half-life; λ is the rate.

- What training does: make the error small, by walking downhill (negative gradient) — gradient descent.
- Question this slide answers: why do different directions learn at different speeds?
- Picture a stretched bowl. In its own tilted coordinates the descent path mixes the two directions → no clean rate for either.
- Fix: use the bowl's own axes. These special directions (eigenvectors) are the ones the dynamics never mixes.
- Along one, the error falls in proportion to what's left (the ODE) — same law as cooling coffee / radioactive half-life → exponential decay, $\alpha(t) = \alpha(0) e^{-\lambda t}$.
- Rate λ = eigenvalue: large is fast, small is slow.
- Everything after works out these directions and rates for a real network on the circle.

Each direction has its own rate

Training makes the error small by walking downhill, along the steepest drop.

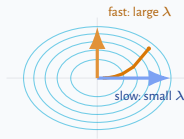


In x, y axes the two directions are coupled.

Along one direction, the error falls in proportion to how much is left:

$$\frac{d}{dt} \alpha(t) = -\lambda \alpha(t)$$

⇒
the bowl's own
axes

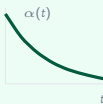


Along each axis the error falls on its own.

so it decays exponentially:

$$\alpha(t) = \alpha(0) e^{-\lambda t}$$

like cooling or half-life; λ is the rate.



Eigenvector: a direction the dynamics keeps separate, never mixing it with others. **Eigenvalue λ :** its rate, large is fast, small is slow.

What are these directions and rates for a neural network?

recall the loss: $\mathcal{L}(f) = \frac{1}{n} \sum_i (f(x_i) - f^*(x_i))^2$

2026-06-11

background

Each direction has its own rate

Training makes the error small by walking downhill, along the steepest drop.



- What training does: make the error small, by walking downhill (negative gradient) — gradient descent.
- Question this slide answers: why do different directions learn at different speeds?
- Picture a stretched bowl. In its own tilted coordinates the descent path mixes the two directions → no clean rate for either.
- Fix: use the bowl's own axes. These special directions (eigenvectors) are the ones the dynamics never mixes.
- Along one, the error falls in proportion to what's left (the ODE) — same law as cooling coffee / radioactive half-life → exponential decay, $\alpha(t) = \alpha(0)e^{-\lambda t}$.
- Rate λ = eigenvalue: large is fast, small is slow.
- Everything after works out these directions and rates for a real network on the circle.

The rate becomes an operator: the Neural Tangent Kernel (Jacot et al., 2018)

The same law as before, with the single rate replaced by an operator.

recall the residual: $r(x) = f(x) - f^*(x)$

One direction

$$\frac{d}{dt}\alpha(t) = -\lambda \alpha(t)$$

The error falls at rate λ , a single number.

2026-06-11

- Picking up the question from the last slide.
- Single direction: $\dot{\alpha} = -\lambda \alpha$, the rate was one number.
- For a network the error is a whole function (the residual); still falls in proportion to what's left, but the proportion is no longer a number.
- It's an operator on the whole function — the neural tangent kernel: $\dot{r} = -Tr$.
- Holds for any differentiable model under gradient flow — why I haven't written the network down yet.
- If T were fixed, diagonalizing splits it back into one scalar law per eigenfunction: $\dot{\alpha}_k = -\lambda_k \alpha_k$.
- So eigenfunctions = directions, eigenvalues = rates. The question becomes: what is the spectrum of T ?

The rate becomes an operator: the Neural Tangent Kernel (Jacot et al., 2018)

The same law as before, with the single rate replaced by an operator.

recall the residual: $r(x) = f(x) - f^*(x)$

One direction

$$\frac{d}{dt}\alpha(t) = -\lambda\alpha(t)$$

The error falls at rate λ , a single number.

A whole function

$$\partial_t r_t = -T_t r_t$$

The rate becomes an operator T_t , the network's tangent kernel (NTK) operator.

2026-06-11

The rate becomes an operator: the Neural Tangent Kernel (Jacot et al., 2018)

The same law as before, with the single rate replaced by an operator

recall the residual: $r(x) = f(x) - f^*(x)$

One direction

$$\frac{d}{dt}\alpha(t) = -\lambda\alpha(t)$$

The error falls at rate λ , a single number

A whole function

$$\partial_t r_t = -T_t r_t$$

The rate becomes an operator T_t , the network's tangent kernel (NTK) operator

- Picking up the question from the last slide.
- Single direction: $\dot{\alpha} = -\lambda\alpha$, the rate was one number.
- For a network the error is a whole function (the residual); still falls in proportion to what's left, but the proportion is no longer a number.
- It's an operator on the whole function — the neural tangent kernel: $\dot{r} = -Tr$.
- Holds for any differentiable model under gradient flow — why I haven't written the network down yet.
- If T were fixed, diagonalizing splits it back into one scalar law per eigenfunction: $\dot{\alpha}_k = -\lambda_k\alpha_k$.
- So eigenfunctions = directions, eigenvalues = rates. The question becomes: what is the spectrum of T ?

The rate becomes an operator: the Neural Tangent Kernel (Jacot et al., 2018)

The same law as before, with the single rate replaced by an operator.

recall the residual: $r(x) = f(x) - f^*(x)$

One direction

$$\frac{d}{dt}\alpha(t) = -\lambda\alpha(t)$$

The error falls at rate λ , a single number.

A whole function

$$\partial_t r_t = -T_t r_t$$

The rate becomes an operator T_t , the network's tangent kernel (NTK) operator.

If T_t were time-independent, diagonalizing it would split this back into one copy of the scalar law per direction k :

$$\dot{\alpha}_k = -\lambda_k \alpha_k$$

eigenfunctions are the directions · **eigenvalues** λ_k are the rates · together they form the **spectrum** of the time-independent operator T

2026-06-11

- Picking up the question from the last slide.
- Single direction: $\dot{\alpha} = -\lambda\alpha$, the rate was one number.
- For a network the error is a whole function (the residual); still falls in proportion to what's left, but the proportion is no longer a number.
- It's an operator on the whole function — the neural tangent kernel: $\dot{r} = -Tr$.
- Holds for any differentiable model under gradient flow — why I haven't written the network down yet.
- If T were fixed, diagonalizing splits it back into one scalar law per eigenfunction: $\dot{\alpha}_k = -\lambda_k \alpha_k$.
- So eigenfunctions = directions, eigenvalues = rates. The question becomes: what is the spectrum of T ?



The rate becomes an operator: the Neural Tangent Kernel (Jacot et al., 2018)

The same law as before, with the single rate replaced by an operator.

recall the residual: $r(x) = f(x) - f^*(x)$

One direction

$$\frac{d}{dt}\alpha(t) = -\lambda\alpha(t)$$

The error falls at rate λ , a single number.

A whole function

$$\partial_t r_t = -T_t r_t$$

The rate becomes an operator T_t , the network's tangent kernel (NTK) operator.

If T_t were time-independent, diagonalizing it would split this back into one copy of the scalar law per direction k :

$$\dot{\alpha}_k = -\lambda_k \alpha_k$$

eigenfunctions are the directions · **eigenvalues** λ_k are the rates · together they form the **spectrum** of the time-independent operator T

So the question becomes: what is the spectrum of T ?

2026-06-11

- Picking up the question from the last slide.
- Single direction: $\dot{\alpha} = -\lambda\alpha$, the rate was one number.
- For a network the error is a whole function (the residual); still falls in proportion to what's left, but the proportion is no longer a number.
- It's an operator on the whole function — the neural tangent kernel: $\dot{r} = -Tr$.
- Holds for any differentiable model under gradient flow — why I haven't written the network down yet.
- If T were fixed, diagonalizing splits it back into one scalar law per eigenfunction: $\dot{\alpha}_k = -\lambda_k \alpha_k$.
- So eigenfunctions = directions, eigenvalues = rates. The question becomes: what is the spectrum of T ?

The rate becomes an operator: the Neural Tangent Kernel (Jacot et al., 2018)

The same law as before, with the single rate replaced by an operator

recall the residual: $r(x) = f(x) - f^*(x)$

One direction $\frac{d}{dt}\alpha(t) = -\lambda\alpha(t)$ The error falls at rate λ, a single number	A whole function $\partial_t r_t = -T_t r_t$ The rate becomes an operator T_t, the network's tangent kernel (NTK) operator
--	--

If T_t were time-independent, diagonalizing it would split this back into one copy of the scalar law per direction k :

eigenfunctions are the directions · **eigenvalues** λ_k are the rates · together they form the **spectrum** of the time-independent operator T

So the question becomes: what is the spectrum of T ?

Making the dynamics analyzable

To read off a fixed set of directions and rates, T must be one fixed operator. Two things are in the way.

recall: $\partial_t r_t = -T_t r_t$

Time-dependent

T_t changes as the network trains, so its directions and rates drift.

2026-06-11

- To read off fixed directions and rates, T must be one fixed operator — and it isn't yet. Two obstacles.
- Time-dependent: T changes as the network trains, so directions/rates drift → freeze it to a constant operator.
- Scale-dependent: at finite data depends on the sampled points; at finite width random (depends on init) → take continuum + infinite width → a single deterministic operator with exact structure.
- Strategy: analyze this clean reference first (exact spectrum), then relax each idealization separately.
- Finite data = one perturbation; finite width = the other (first frozen kernel, then evolving).

Time-dependent

T_t changes as the network trains, so its directions and rates drift.

Making the dynamics analyzable

To read off a fixed set of directions and rates, T must be one fixed operator. Two things are in the way.

$$\text{recall: } \partial_t r_t = -T_t r_t$$

Time-dependent

T_t changes as the network trains, so its directions and rates drift.

Solution: Take width to infinity to get a constant operator T (Jacot et al., 2018).

2026-06-11

To read off a fixed set of directions and rates, T must be one fixed operator. Two things are in the way. with $\Delta_{\mathbf{A}_t} = \mathbf{A}_t - \mathbf{A}_0$

Time-dependent

T_t changes as the network trains, so its directions and rates drift.

Solution: Take width to infinity to get a constant operator T (Jacot et al., 2018).

- To read off fixed directions and rates, T must be one fixed operator — and it isn't yet. Two obstacles.
- Time-dependent: T changes as the network trains, so directions/rates drift $\rightarrow$ freeze it to a constant operator.
- Scale-dependent: at finite data depends on the sampled points; at finite width random (depends on init) $\rightarrow$ take continuum + infinite width $\rightarrow$ a single deterministic operator with exact structure.
- Strategy: analyze this clean reference first (exact spectrum), then relax each idealization separately.
- Finite data = one perturbation; finite width = the other (first frozen kernel, then evolving).

Making the dynamics analyzable

To read off a fixed set of directions and rates, T must be one fixed operator. Two things are in the way.

$$\text{recall: } \partial_t r_t = -T_t r_t$$

Time-dependent

T_t changes as the network trains, so its directions and rates drift.

Solution: Take width to infinity to get a constant operator T (Jacot et al., 2018).

Finite scale

At finite data T depends on the sampled input points, so it changes on what it sees.

2026-06-11

- To read off fixed directions and rates, T must be one fixed operator — and it isn't yet. Two obstacles.
- Time-dependent: T changes as the network trains, so directions/rates drift → freeze it to a constant operator.
- Scale-dependent: at finite data depends on the sampled points; at finite width random (depends on init) → take continuum + infinite width → a single deterministic operator with exact structure.
- Strategy: analyze this clean reference first (exact spectrum), then relax each idealization separately.
- Finite data = one perturbation; finite width = the other (first frozen kernel, then evolving).

Making the dynamics analyzable

To read off a fixed set of directions and rates, T must be one fixed operator. Two things are in the way.

Time-dependent

T_t changes as the network trains, so its directions and rates drift.

Solution: Take width to infinity to get a constant operator T (Jacot et al., 2018).

Finite scale

At finite data T depends on the sampled input points, so it changes on what it sees.

Making the dynamics analyzable

To read off a fixed set of directions and rates, T must be one fixed operator. Two things are in the way.

recall: $\partial_t r_t = -T_t r_t$

Time-dependent

T_t changes as the network trains, so its directions and rates drift.

Solution: Take width to infinity to get a constant operator T (Jacot et al., 2018).

Finite scale

At finite data T depends on the sampled input points, so it changes on what it sees.

Solution: Consider using the continuum as input. (Jacot et al., 2018, Cao et al., 2019).

2026-06-11

- To read off fixed directions and rates, T must be one fixed operator — and it isn't yet. Two obstacles.
- Time-dependent: T changes as the network trains, so directions/rates drift $\rightarrow$ freeze it to a constant operator.
- Scale-dependent: at finite data depends on the sampled points; at finite width random (depends on init) $\rightarrow$ take continuum + infinite width $\rightarrow$ a single deterministic operator with exact structure.
- Strategy: analyze this clean reference first (exact spectrum), then relax each idealization separately.
- Finite data = one perturbation; finite width = the other (first frozen kernel, then evolving).

Making the dynamics analyzable

To read off a fixed set of directions and rates, T must be one fixed operator. Two things are in the way.

Time-dependent

T_t changes as the network trains, so its directions and rates drift.

Solution: Take width to infinity to get a constant operator T (Jacot et al., 2018).

Finite scale

At finite data T depends on the sampled input points, so it changes on what it sees.

Solution: Consider using the continuum as input. (Jacot et al., 2018, Cao et al., 2019).

Making the dynamics analyzable

To read off a fixed set of directions and rates, T must be one fixed operator. Two things are in the way.

recall: $\partial_t r_t = -T_t r_t$

Time-dependent

T_t changes as the network trains, so its directions and rates drift.

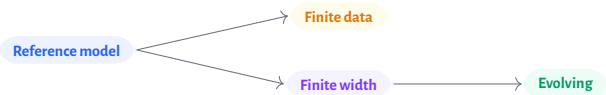
Solution: Take width to infinity to get a constant operator T (Jacot et al., 2018).

Finite scale

At finite data T depends on the sampled input points, so it changes on what it sees.

Solution: Consider using the continuum as input. (Jacot et al., 2018, Cao et al., 2019).

Strategy: build the clean reference, then relax each idealization separately.



2026-06-11

- To read off fixed directions and rates, T must be one fixed operator — and it isn't yet. Two obstacles.
- Time-dependent: T changes as the network trains, so directions/rates drift → freeze it to a constant operator.
- Scale-dependent: at finite data depends on the sampled points; at finite width random (depends on init) → take continuum + infinite width → a single deterministic operator with exact structure.
- Strategy: analyze this clean reference first (exact spectrum), then relax each idealization separately.
- Finite data = one perturbation; finite width = the other (first frozen kernel, then evolving).

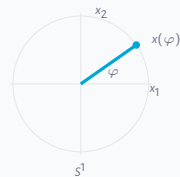


The concrete model: regression on S^1

The unit circle gives a natural language for “coarse” and “fine”: Fourier modes.

- Inputs are points on the unit circle with uniform distribution:

$$x(\varphi) = (\cos \varphi, \sin \varphi) \in S^1.$$



2026-06-11

- Instantiate the abstract operator story.
- Circle gives Fourier modes: coarse to fine.
- Model: one-hidden-layer ReLU network.
- Define ϕ_k as a Fourier mode and track coefficients $\alpha_k(t)$.
- Next: ideal limit makes these exact eigen-directions.



The concrete model: regression on S^1

The unit circle gives a natural language for “coarse” and “fine”: Fourier modes.

- Inputs are points on the unit circle with uniform distribution:

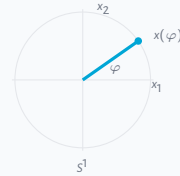
$$x(\varphi) = (\cos \varphi, \sin \varphi) \in S^1.$$

- Fourier basis functions are written compactly as

$$\phi_k \in \{1\} \cup \{\sin(k\varphi), \cos(k\varphi) : k \geq 1\}.$$

- The target is a Fourier sum with amplitudes c_k , the weights of each mode:

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$



2026-06-11

- Instantiate the abstract operator story.
- Circle gives Fourier modes: coarse to fine.
- Model: one-hidden-layer ReLU network.
- Define ϕ_k as a Fourier mode and track coefficients $\alpha_k(t)$.
- Next: ideal limit makes these exact eigen-directions.

The unit circle gives a natural language for “coarse” and “fine”: Fourier modes.

- Inputs are points on the unit circle with uniform distribution:

$$x(\varphi) = (\cos \varphi, \sin \varphi) \in S^1.$$

- Fourier basis functions are written compactly as

$$\phi_k \in \{1\} \cup \{\sin(k\varphi), \cos(k\varphi) : k \geq 1\}.$$

- The target is a Fourier sum with amplitudes c_k , the weights of each mode:

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

The concrete model: regression on S^1

The unit circle gives a natural language for “coarse” and “fine”: Fourier modes.

- Inputs are points on the unit circle with uniform distribution:

$$x(\varphi) = (\cos \varphi, \sin \varphi) \in S^1.$$

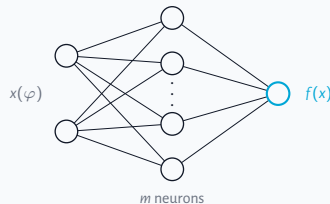
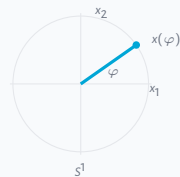
- Fourier basis functions are written compactly as

$$\phi_k \in \{1\} \cup \{\sin(k\varphi), \cos(k\varphi) : k \geq 1\}.$$

- The target is a Fourier sum with amplitudes c_k , the weights of each mode:

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

- The model is a one-hidden-layer ReLU network of width m , initialized with $w_i \sim \mathcal{N}(0, I_2)$, $a_i \sim \mathcal{N}(0, 1)$, and $b_i = 0$.



$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

2026-06-11

- Instantiate the abstract operator story.
- Circle gives Fourier modes: coarse to fine.
- Model: one-hidden-layer ReLU network.
- Define ϕ_k as a Fourier mode and track coefficients $\alpha_k(t)$.
- Next: ideal limit makes these exact eigen-directions.

setup

The concrete model: regression on S^1

The unit circle gives a natural language for “coarse” and “fine”: Fourier modes.

- Inputs are points on the unit circle with uniform distribution:

$$x(\varphi) = (\cos \varphi, \sin \varphi) \in S^1.$$

- Fourier basis functions are written compactly as

$$\phi_k \in \{1\} \cup \{\sin(k\varphi), \cos(k\varphi) : k \geq 1\}.$$

- The target is a Fourier sum with amplitudes c_k , the weights of each mode:

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

- The model is a one-hidden-layer ReLU network of width m , initialized with $w_i \sim \mathcal{N}(0, I_2)$, $a_i \sim \mathcal{N}(0, 1)$, and $b_i = 0$.

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

The concrete model: regression on S^1

The unit circle gives a natural language for “coarse” and “fine”: Fourier modes.

- Inputs are points on the unit circle with uniform distribution:

$$x(\varphi) = (\cos \varphi, \sin \varphi) \in S^1.$$

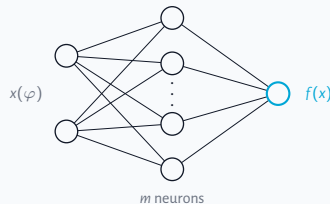
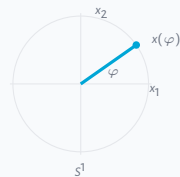
- Fourier basis functions are written compactly as

$$\phi_k \in \{1\} \cup \{\sin(k\varphi), \cos(k\varphi) : k \geq 1\}.$$

- The target is a Fourier sum with amplitudes c_k , the weights of each mode:

$$f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad r_t = f_t - f^* = \sum_k \alpha_k(t) \phi_k.$$

- The model is a one-hidden-layer ReLU network of width m , initialized with $w_i \sim \mathcal{N}(0, I_2)$, $a_i \sim \mathcal{N}(0, 1)$, and $b_i = 0$.
- Question:** what controls the decay of each $\alpha_k(t)$?



$$f_m(x; \theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^m a_i \sigma(w_i^\top x + b_i), \quad \sigma = \text{ReLU}$$

2026-06-11

- Instantiate the abstract operator story.
- Circle gives Fourier modes: coarse to fine.
- Model: one-hidden-layer ReLU network.
- Define ϕ_k as a Fourier mode and track coefficients $\alpha_k(t)$.
- Next: ideal limit makes these exact eigen-directions.

The unit circle gives a natural language for “coarse” and “fine”: Fourier modes.

- Inputs are points on the unit circle with uniform distribution:
 $x(\varphi) = [\cos \varphi, \sin \varphi] \in \mathbb{R}^2$.
- Fourier basis functions are written compactly as:
 $\phi_k \in \{1\} \cup \{\sin(k\varphi), \cos(k\varphi) : k \geq 1\}$.
- The target is a Fourier sum with amplitudes c_k , the weights of each mode:
 $f^*(\varphi) = \sum_k c_k \phi_k(\varphi), \quad c_k \in \mathbb{R}, \quad f^* \in C^\infty \Rightarrow \sum_k |c_k| < \infty$.
- The model is a one-hidden-layer ReLU network of width m , initialized with $w_i \sim \mathcal{N}(0, I_2)$, $a_i \sim \mathcal{N}(0, 1)$, and $b_i = 0$.
- Question: what controls the decay of each $\alpha_k(t)$?



The ideal picture: a convolution operator on S^1

At infinite width and with continuum data, the NTK becomes fixed and rotation-invariant.

recall: fixed T gives $\dot{\alpha}_k = -\lambda_k \alpha_k$

With uniform measure $\mu(d\psi) = d\psi / (2\pi)$, the fixed operator acts by

2026-06-11

- Infinite width and continuum data.
- Uniform circle makes T a convolution operator.
- Fourier modes diagonalize convolution.
- Each mode decays independently at rate λ_k .
- Formula shows $\lambda_k \sim k^{-2}$: high frequencies are slow.

The ideal picture: a convolution operator on S^1

At infinite width and with continuum data, the NTK becomes fixed and rotation-invariant.

recall: fixed T gives $\dot{\alpha}_k = -\lambda_k \alpha_k$

With uniform measure $\mu(d\psi) = d\psi / (2\pi)$, the fixed operator acts by

$$(Tr)(\varphi) = \int_0^{2\pi} \Theta(\varphi - \psi) r(\psi) \frac{d\psi}{2\pi}.$$

2026-06-11

- Infinite width and continuum data.
- Uniform circle makes T a convolution operator.
- Fourier modes diagonalize convolution.
- Each mode decays independently at rate λ_k .
- Formula shows $\lambda_k \sim k^{-2}$: high frequencies are slow.

With uniform measure $\mu(d\psi) = d\psi / (2\pi)$, the fixed operator acts by

$$(Tr)(\varphi) = \int_0^{2\pi} \Theta(\varphi - \psi) r(\psi) \frac{d\psi}{2\pi}.$$

The ideal picture: a convolution operator on S^1

At infinite width and with continuum data, the NTK becomes fixed and rotation-invariant.

recall: fixed T gives $\dot{\alpha}_k = -\lambda_k \alpha_k$

With uniform measure $\mu(d\psi) = d\psi / (2\pi)$, the fixed operator acts by

$$(Tr)(\varphi) = \int_0^{2\pi} \Theta(\varphi - \psi) r(\psi) \frac{d\psi}{2\pi}.$$

$$\Theta(\varphi, \psi) = \langle \nabla_{\theta} f(x(\varphi)), \nabla_{\theta} f(x(\psi)) \rangle$$

2026-06-11

- Infinite width and continuum data.
- Uniform circle makes T a convolution operator.
- Fourier modes diagonalize convolution.
- Each mode decays independently at rate λ_k .
- Formula shows $\lambda_k \sim k^{-2}$: high frequencies are slow.

$$(Tr)(\varphi) = \int_0^{2\pi} \Theta(\varphi - \psi) r(\psi) \frac{d\psi}{2\pi}.$$

The ideal picture: a convolution operator on S^1

At infinite width and with continuum data, the NTK becomes fixed and rotation-invariant.

recall: fixed T gives $\dot{\alpha}_k = -\lambda_k \alpha_k$

With uniform measure $\mu(d\psi) = d\psi / (2\pi)$, the fixed operator acts by

$$(Tr)(\varphi) = \int_0^{2\pi} \Theta(\varphi - \psi) r(\psi) \frac{d\psi}{2\pi}.$$

$$\Theta(\varphi, \psi) = \langle \nabla_{\theta} f(x(\varphi)), \nabla_{\theta} f(x(\psi)) \rangle$$

Because this is a convolution on the circle, the Fourier modes are eigenfunctions:

2026-06-11

- Infinite width and continuum data.
- Uniform circle makes T a convolution operator.
- Fourier modes diagonalize convolution.
- Each mode decays independently at rate λ_k .
- Formula shows $\lambda_k \sim k^{-2}$: high frequencies are slow.

The ideal picture: a convolution operator on S^1

At infinite width and with continuum data, the NTK becomes fixed and rotation-invariant.

recall: fixed T gives $\dot{\alpha}_k = -\lambda_k \alpha_k$

With uniform measure $\mu(d\psi) = d\psi / (2\pi)$, the fixed operator acts by

$$(Tr)(\varphi) = \int_0^{2\pi} \Theta(\varphi - \psi) r(\psi) \frac{d\psi}{2\pi}.$$

$$\Theta(\varphi, \psi) = \langle \nabla_{\theta} f(x(\varphi)), \nabla_{\theta} f(x(\psi)) \rangle$$

Because this is a convolution on the circle, the Fourier modes are eigenfunctions:

$$T\phi_k = \lambda_k \phi_k, \quad \alpha_k(t) = e^{-\lambda_k t} \alpha_k(0).$$

2026-06-11

- Infinite width and continuum data.
- Uniform circle makes T a convolution operator.
- Fourier modes diagonalize convolution.
- Each mode decays independently at rate λ_k .
- Formula shows $\lambda_k \sim k^{-2}$: high frequencies are slow.

With uniform measure $\mu(d\psi) = d\psi / (2\pi)$, the fixed operator acts by

$$(Tr)(\varphi) = \int_0^{2\pi} \Theta(\varphi - \psi) r(\psi) \frac{d\psi}{2\pi}.$$

Because this is a convolution on the circle, the Fourier modes are eigenfunctions:

$$T\phi_k = \lambda_k \phi_k, \quad \alpha_k(t) = e^{-\lambda_k t} \alpha_k(0).$$

The ideal picture: a convolution operator on S^1

At infinite width and with continuum data, the NTK becomes fixed and rotation-invariant.

recall: fixed T gives $\dot{\alpha}_k = -\lambda_k \alpha_k$

With uniform measure $\mu(d\psi) = d\psi / (2\pi)$, the fixed operator acts by

$$(Tr)(\varphi) = \int_0^{2\pi} \Theta(\varphi - \psi) r(\psi) \frac{d\psi}{2\pi}.$$

$$\Theta(\varphi, \psi) = \langle \nabla_{\theta} f(x(\varphi)), \nabla_{\theta} f(x(\psi)) \rangle$$

Because this is a convolution on the circle, the Fourier modes are eigenfunctions:

$$T\phi_k = \lambda_k \phi_k, \quad \alpha_k(t) = e^{-\lambda_k t} \alpha_k(0).$$

Explicit eigenvalues

Thm. 5.2

$$\lambda_k = \begin{cases} \frac{1}{4} + \frac{3}{\pi^2}, & k = 0, \\ \frac{1}{4} + \frac{1}{\pi^2}, & k = 1, \\ \frac{k^2 + 3}{\pi^2(k^2 - 1)^2}, & k \geq 2 \text{ even}, \\ \frac{1}{\pi^2 k^2}, & k \geq 3 \text{ odd}. \end{cases}$$

Jacot et al., 2018; Cao et al., 2019

2026-06-11

- Infinite width and continuum data.
- Uniform circle makes T a convolution operator.
- Fourier modes diagonalize convolution.
- Each mode decays independently at rate λ_k .
- Formula shows $\lambda_k \sim k^{-2}$: high frequencies are slow.

lecture notes

The ideal picture: a convolution operator on S^1

At infinite width and with continuum data, the NTK becomes fixed and rotation-invariant.

With uniform measure $\mu(d\psi) = d\psi / (2\pi)$, the fixed operator acts by

$$(Tr)(\varphi) = \int_0^{2\pi} \Theta(\varphi - \psi) r(\psi) \frac{d\psi}{2\pi}$$

Because this is a convolution on the circle, the Fourier modes are eigenfunctions:

$$T\phi_k = \lambda_k \phi_k, \quad \alpha_k(t) = e^{-\lambda_k t} \alpha_k(0)$$

Explicit eigenvalues

$$\lambda_k = \begin{cases} \frac{1}{4} + \frac{3}{\pi^2}, & k = 0, \\ \frac{1}{4} + \frac{1}{\pi^2}, & k = 1, \\ \frac{k^2 + 3}{\pi^2(k^2 - 1)^2}, & k \geq 2 \text{ even}, \\ \frac{1}{\pi^2 k^2}, & k \geq 3 \text{ odd}. \end{cases}$$

Source: [1], [2], [3], [4], [5]

The ideal picture: a convolution operator on S^1

At infinite width and with continuum data, the NTK becomes fixed and rotation-invariant.

recall: fixed T gives $\dot{\alpha}_k = -\lambda_k \alpha_k$

With uniform measure $\mu(d\psi) = d\psi / (2\pi)$, the fixed operator acts by

$$(Tr)(\varphi) = \int_0^{2\pi} \Theta(\varphi - \psi) r(\psi) \frac{d\psi}{2\pi}.$$

$$\Theta(\varphi, \psi) = \langle \nabla_{\theta} f(x(\varphi)), \nabla_{\theta} f(x(\psi)) \rangle$$

Because this is a convolution on the circle, the Fourier modes are eigenfunctions:

$$T\phi_k = \lambda_k \phi_k, \quad \alpha_k(t) = e^{-\lambda_k t} \alpha_k(0).$$

Explicit eigenvalues

Thm. 5.2

$$\lambda_k = \begin{cases} \frac{1}{4} + \frac{3}{\pi^2}, & k = 0, \\ \frac{1}{4} + \frac{1}{\pi^2}, & k = 1, \\ \frac{k^2+3}{\pi^2(k^2-1)^2}, & k \geq 2 \text{ even}, \\ \frac{1}{\pi^2 k^2}, & k \geq 3 \text{ odd}. \end{cases}$$

Interpretation

- ▶ Low frequencies $\implies$ Small k : larger λ_k , faster decay.
- ▶ High frequencies $\implies$ Large k : $\lambda_k \sim k^{-2}$, slower decay.

2026-06-11

The ideal picture: a convolution operator on S^1

At infinite width and with continuum data, the NTK becomes fixed and rotation-invariant.

With uniform measure $\mu(d\psi) = d\psi / (2\pi)$, the fixed operator acts by

$$(Tr)(\varphi) = \int_0^{2\pi} \Theta(\varphi - \psi) r(\psi) \frac{d\psi}{2\pi}.$$

Because this is a convolution on the circle, the Fourier modes are eigenfunctions:

$$T\phi_k = \lambda_k \phi_k, \quad \alpha_k(t) = e^{-\lambda_k t} \alpha_k(0).$$

Explicit eigenvalues

$$\lambda_k = \begin{cases} \frac{1}{4} + \frac{3}{\pi^2}, & k = 0, \\ \frac{1}{4} + \frac{1}{\pi^2}, & k = 1, \\ \frac{k^2+3}{\pi^2(k^2-1)^2}, & k \geq 2 \text{ even}, \\ \frac{1}{\pi^2 k^2}, & k \geq 3 \text{ odd}. \end{cases}$$

Interpretation

- ▶ Low frequencies $\implies$ Small k : larger λ_k , faster decay.
- ▶ High frequencies $\implies$ Large k : $\lambda_k \sim k^{-2}$, slower decay.

- Infinite width and continuum data.
- Uniform circle makes T a convolution operator.
- Fourier modes diagonalize convolution.
- Each mode decays independently at rate λ_k .
- Formula shows $\lambda_k \sim k^{-2}$: high frequencies are slow.

The ideal picture: a convolution operator on S^1

At infinite width and with continuum data, the NTK becomes fixed and rotation-invariant.

recall: fixed T gives $\dot{\alpha}_k = -\lambda_k \alpha_k$

With uniform measure $\mu(d\psi) = d\psi / (2\pi)$, the fixed operator acts by

$$(Tr)(\varphi) = \int_0^{2\pi} \Theta(\varphi - \psi) r(\psi) \frac{d\psi}{2\pi}.$$

$$\Theta(\varphi, \psi) = \langle \nabla_{\theta} f(x(\varphi)), \nabla_{\theta} f(x(\psi)) \rangle$$

Because this is a convolution on the circle, the Fourier modes are eigenfunctions:

$$T\phi_k = \lambda_k \phi_k, \quad \alpha_k(t) = e^{-\lambda_k t} \alpha_k(0).$$

Explicit eigenvalues

Thm. 5.2

$$\lambda_k = \begin{cases} \frac{1}{4} + \frac{3}{\pi^2}, & k = 0, \\ \frac{1}{4} + \frac{1}{\pi^2}, & k = 1, \\ \frac{k^2+3}{\pi^2(k^2-1)^2}, & k \geq 2 \text{ even}, \\ \frac{1}{\pi^2 k^2}, & k \geq 3 \text{ odd}. \end{cases}$$

Interpretation

- ▶ Low frequencies $\implies$ Small k : larger λ_k , faster decay.
- ▶ High frequencies $\implies$ Large k : $\lambda_k \sim k^{-2}$, slower decay.

This answers the question of why coarse structure first, fine detail later.

2026-06-11



- Infinite width and continuum data.
- Uniform circle makes T a convolution operator.
- Fourier modes diagonalize convolution.
- Each mode decays independently at rate λ_k .
- Formula shows $\lambda_k \sim k^{-2}$: high frequencies are slow.

Does this spectral picture survive outside the ideal limit?

The ideal model is useful only if we understand what happens when its assumptions are relaxed.

2026-06-11

- The ideal model is useful only if we understand what happens when its assumptions are relaxed.
- Perturbation 1: finite samples.
- Replace continuum measure by random sample points:

$$\mu \rightsquigarrow \frac{1}{n} \sum_i \delta_{\varphi_i}.$$

- This breaks exact Fourier orthogonality and exact diagonalization.
- Question: do Fourier modes still behave approximately like eigenvectors of the sampled operator?
- Perturbation 2: frozen finite width.
- Replace deterministic T by random finite-width tangent operator $T_0^{(m)}$.
- Question: are Fourier eigenspaces and learning rates preserved?
- Main theoretical part: control both finite data and frozen finite width.

Does this spectral picture survive outside the ideal limit?

The ideal model is useful only if we understand what happens when its assumptions are relaxed.

Perturbation I

Finite samples

$$\mu \rightsquigarrow \frac{1}{n} \sum_{i=1}^n \delta_{\varphi_i}$$

Replace the continuous measure by n random points on S^1 .

2026-06-11

thesis question

Does this spectral picture survive outside the ideal limit?

The ideal model is useful only if we understand what happens when its assumptions are relaxed.

Perturbation I

Finite samples

$$\mu \rightsquigarrow \frac{1}{n} \sum_{i=1}^n \delta_{\varphi_i}$$

Replace the continuous measure by n random points on S^1 .

- The ideal model is useful only if we understand what happens when its assumptions are relaxed.
- Perturbation 1: finite samples.
- Replace continuum measure by random sample points:

$$\mu \rightsquigarrow \frac{1}{n} \sum_i \delta_{\varphi_i}.$$

- This breaks exact Fourier orthogonality and exact diagonalization.
- Question: do Fourier modes still behave approximately like eigenvectors of the sampled operator?
- Perturbation 2: frozen finite width.
- Replace deterministic T by random finite-width tangent operator $T_0^{(m)}$.
- Question: are Fourier eigenspaces and learning rates preserved?
- Main theoretical part: control both finite data and frozen finite width.

Does this spectral picture survive outside the ideal limit?

The ideal model is useful only if we understand what happens when its assumptions are relaxed.

Perturbation I

Finite samples

$$\mu \rightsquigarrow \frac{1}{n} \sum_{i=1}^n \delta_{\varphi_i}$$

Replace the continuous measure by n random points on S^1 .

Question

Do Fourier modes still behave like eigenvectors of the sampled operator?

2026-06-11

- The ideal model is useful only if we understand what happens when its assumptions are relaxed.
- Perturbation 1: finite samples.
- Replace continuum measure by random sample points:

$$\mu \rightsquigarrow \frac{1}{n} \sum_i \delta_{\varphi_i}.$$

- This breaks exact Fourier orthogonality and exact diagonalization.
- Question: do Fourier modes still behave approximately like eigenvectors of the sampled operator?
- Perturbation 2: frozen finite width.
- Replace deterministic T by random finite-width tangent operator $T_0^{(m)}$.
- Question: are Fourier eigenspaces and learning rates preserved?
- Main theoretical part: control both finite data and frozen finite width.

Perturbation I
Finite samples

$$\mu \rightsquigarrow \frac{1}{n} \sum_{i=1}^n \delta_{\varphi_i}$$

Replace the continuous measure by n random points on S^1 .

Question
 Do Fourier modes still behave like eigenvectors of the sampled operator?

Does this spectral picture survive outside the ideal limit?

The ideal model is useful only if we understand what happens when its assumptions are relaxed.

Perturbation I

Finite samples

$$\mu \rightsquigarrow \frac{1}{n} \sum_{i=1}^n \delta_{\varphi_i}$$

Replace the continuous measure by n random points on S^1 .

Question

Do Fourier modes still behave like eigenvectors of the sampled operator?

vs

Perturbation II

Finite width

$$T \rightsquigarrow T^{(m)}$$

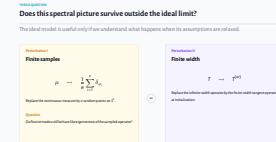
Replace the infinite-width operator by the finite-width tangent operator at initialization.

2026-06-11

- The ideal model is useful only if we understand what happens when its assumptions are relaxed.
- Perturbation 1: finite samples.
- Replace continuum measure by random sample points:

$$\mu \rightsquigarrow \frac{1}{n} \sum_i \delta_{\varphi_i}.$$

- This breaks exact Fourier orthogonality and exact diagonalization.
- Question: do Fourier modes still behave approximately like eigenvectors of the sampled operator?
- Perturbation 2: frozen finite width.
- Replace deterministic T by random finite-width tangent operator $T_0^{(m)}$.
- Question: are Fourier eigenspaces and learning rates preserved?
- Main theoretical part: control both finite data and frozen finite width.



Does this spectral picture survive outside the ideal limit?

The ideal model is useful only if we understand what happens when its assumptions are relaxed.

Perturbation I

Finite samples

$$\mu \rightsquigarrow \frac{1}{n} \sum_{i=1}^n \delta_{\varphi_i}$$

Replace the continuous measure by n random points on S^1 .

Question

Do Fourier modes still behave like eigenvectors of the sampled operator?

vs

Perturbation II

Finite width

$$T \rightsquigarrow T^{(m)}$$

Replace the infinite-width operator by the finite-width tangent operator at initialization.

Question

Does the finite-width tangent operator preserve the Fourier eigenspaces and learning rates?

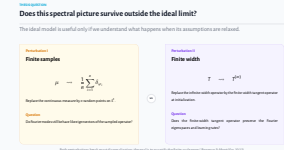
Both perturbations break exact diagonalization; the goal is to quantify the finite-scale error (Bowman & Montúfar, 2022)

2026-06-11

- The ideal model is useful only if we understand what happens when its assumptions are relaxed.
- Perturbation 1: finite samples.
- Replace continuum measure by random sample points:

$$\mu \rightsquigarrow \frac{1}{n} \sum_i \delta_{\varphi_i}.$$

- This breaks exact Fourier orthogonality and exact diagonalization.
- Question: do Fourier modes still behave approximately like eigenvectors of the sampled operator?
- Perturbation 2: frozen finite width.
- Replace deterministic T by random finite-width tangent operator $T_0^{(m)}$.
- Question: are Fourier eigenspaces and learning rates preserved?
- Main theoretical part: control both finite data and frozen finite width.



Main results: the spectral picture survives finite scale

$$\mathcal{H}_K = \text{span}\{1, \cos \varphi, \sin \varphi, \dots, \cos(K\varphi), \sin(K\varphi)\}, \quad d = 2K + 1.$$

$$\Lambda_K = \text{diag}(\lambda_0, \lambda_1, \lambda_1, \dots, \lambda_K, \lambda_K), \text{ and } T_K \text{ denotes the relevant finite-scale operator on } \mathcal{H}_K.$$

2026-06-11

- Fix one low-frequency Fourier subspace.
- On that subspace, the ideal operator is diagonal.
- Finite samples have an exponential tail in n .
- Frozen finite width is unbiased and has an exponential tail in m .
- Detailed Gram and action bounds are in backup.

$$\mathcal{H}_K = \text{span}\{1, \cos \varphi, \sin \varphi, \dots, \cos(K\varphi), \sin(K\varphi)\}, \quad d = 2K + 1.$$

$$\Lambda_K = \text{diag}(\lambda_0, \lambda_1, \lambda_1, \dots, \lambda_K, \lambda_K) \text{ and } T_K \text{ denotes the relevant finite-scale operator on } \mathcal{H}_K.$$

Main results: the spectral picture survives finite scale

$\mathcal{H}_K = \text{span}\{1, \cos \varphi, \sin \varphi, \dots, \cos(K\varphi), \sin(K\varphi)\}$, $d = 2K + 1$.
 $\Lambda_K = \text{diag}(\lambda_0, \lambda_1, \lambda_1, \dots, \lambda_K, \lambda_K)$, and T_K denotes the relevant finite-scale operator on $\mathcal{H}_K$.

Finite samples

Thm. 5.5

Let T_K be the sampled operator on $\mathcal{H}_K$, using n uniform samples from $[0, 2\pi)$.

For every $\varepsilon > 0$,

2026-06-11

- Fix one low-frequency Fourier subspace.
- On that subspace, the ideal operator is diagonal.
- Finite samples have an exponential tail in n .
- Frozen finite width is unbiased and has an exponential tail in m .
- Detailed Gram and action bounds are in backup.

Main results: the spectral picture survives finite scale

$\mathcal{H}_K = \text{span}\{1, \cos \varphi, \sin \varphi, \dots, \cos(K\varphi), \sin(K\varphi)\}$, $d = 2K + 1$.
 $\Lambda_K = \text{diag}(\lambda_0, \lambda_1, \lambda_1, \dots, \lambda_K, \lambda_K)$ and T_K denotes the relevant finite-scale operator on $\mathcal{H}_K$.

Finite samples

Let T_K be the sampled operator on $\mathcal{H}_K$, using n uniform samples from $[0, 2\pi)$.

For every $\varepsilon > 0$,

Main results: the spectral picture survives finite scale

$$\mathcal{H}_K = \text{span}\{1, \cos \varphi, \sin \varphi, \dots, \cos(K\varphi), \sin(K\varphi)\}, \quad d = 2K + 1.$$

$$\Lambda_K = \text{diag}(\lambda_0, \lambda_1, \lambda_1, \dots, \lambda_K, \lambda_K), \text{ and } T_K \text{ denotes the relevant finite-scale operator on } \mathcal{H}_K.$$

Finite samples

Thm. 5.5

Let T_K be the sampled operator on $\mathcal{H}_K$, using n uniform samples from $[0, 2\pi)$.

For every $\varepsilon > 0$,

$$\mathbb{P}(\|T_K - \Lambda_K\|_{\max} \geq \varepsilon) \leq 2d^2 \exp(-Cn\varepsilon^2).$$

2026-06-11

- Fix one low-frequency Fourier subspace.
- On that subspace, the ideal operator is diagonal.
- Finite samples have an exponential tail in n .
- Frozen finite width is unbiased and has an exponential tail in m .
- Detailed Gram and action bounds are in backup.

Main results: the spectral picture survives finite scale

$$\mathcal{H}_K = \text{span}\{1, \cos \varphi, \sin \varphi, \dots, \cos(K\varphi), \sin(K\varphi)\}, \quad d = 2K + 1.$$

$$\Lambda_K = \text{diag}(\lambda_0, \lambda_1, \lambda_1, \dots, \lambda_K, \lambda_K) \text{ and } T_K \text{ denotes the relevant finite-scale operator on } \mathcal{H}_K.$$

Finite samples

Let T_K be the sampled operator on $\mathcal{H}_K$, using n uniform samples from $[0, 2\pi)$.

For every $\varepsilon > 0$,

$$\mathbb{P}(\|T_K - \Lambda_K\|_{\max} \geq \varepsilon) \leq 2d^2 \exp(-Cn\varepsilon^2).$$

Main results: the spectral picture survives finite scale

$$\mathcal{H}_K = \text{span}\{1, \cos \varphi, \sin \varphi, \dots, \cos(K\varphi), \sin(K\varphi)\}, \quad d = 2K + 1.$$

$$\Lambda_K = \text{diag}(\lambda_0, \lambda_1, \lambda_1, \dots, \lambda_K, \lambda_K), \text{ and } T_K \text{ denotes the relevant finite-scale operator on } \mathcal{H}_K.$$

Finite samples

Thm. 5.5

Let T_K be the sampled operator on $\mathcal{H}_K$, using n uniform samples from $[0, 2\pi)$.

For every $\varepsilon > 0$,

$$\mathbb{P}(\|T_K - \Lambda_K\|_{\max} \geq \varepsilon) \leq 2d^2 \exp(-Cn\varepsilon^2).$$

Equivalently, with probability at least $1 - \delta$,

2026-06-11

- Fix one low-frequency Fourier subspace.
- On that subspace, the ideal operator is diagonal.
- Finite samples have an exponential tail in n .
- Frozen finite width is unbiased and has an exponential tail in m .
- Detailed Gram and action bounds are in backup.

Main results: the spectral picture survives finite scale

$\mathcal{H}_K = \text{span}\{1, \cos \varphi, \sin \varphi, \dots, \cos(K\varphi), \sin(K\varphi)\}, \quad d = 2K + 1.$
 $\Lambda_K = \text{diag}(\lambda_0, \lambda_1, \lambda_1, \dots, \lambda_K, \lambda_K)$ and T_K denotes the relevant finite-scale operator on $\mathcal{H}_K$.

Finite samples

Thm. 5.5

Let T_K be the sampled operator on $\mathcal{H}_K$, using n uniform samples from $[0, 2\pi)$.

For every $\varepsilon > 0$,

$$\mathbb{P}(\|T_K - \Lambda_K\|_{\max} \geq \varepsilon) \leq 2d^2 \exp(-Cn\varepsilon^2).$$

Equivalently, with probability at least $1 - \delta$,

Main results: the spectral picture survives finite scale

$$\mathcal{H}_K = \text{span}\{1, \cos \varphi, \sin \varphi, \dots, \cos(K\varphi), \sin(K\varphi)\}, \quad d = 2K + 1.$$

$$\Lambda_K = \text{diag}(\lambda_0, \lambda_1, \lambda_1, \dots, \lambda_K, \lambda_K), \text{ and } T_K \text{ denotes the relevant finite-scale operator on } \mathcal{H}_K.$$

Finite samples

Thm. 5.5

Let T_K be the sampled operator on $\mathcal{H}_K$, using n uniform samples from $[0, 2\pi)$.

For every $\varepsilon > 0$,

$$\mathbb{P}(\|T_K - \Lambda_K\|_{\max} \geq \varepsilon) \leq 2d^2 \exp(-Cn\varepsilon^2).$$

Equivalently, with probability at least $1 - \delta$,

$$\|T_K - \Lambda_K\|_{\max} = O\left(\sqrt{\frac{\log(d/\delta)}{n}}\right).$$

2026-06-11

- Fix one low-frequency Fourier subspace.
- On that subspace, the ideal operator is diagonal.
- Finite samples have an exponential tail in n .
- Frozen finite width is unbiased and has an exponential tail in m .
- Detailed Gram and action bounds are in backup.

Main results: the spectral picture survives finite scale

Let $\mathcal{H}_K = \text{span}\{1, \cos \varphi, \sin \varphi, \dots, \cos(K\varphi), \sin(K\varphi)\}$, $d = 2K + 1$.
 $\Lambda_K = \text{diag}(\lambda_0, \lambda_1, \lambda_1, \dots, \lambda_K, \lambda_K)$ and T_K denotes the relevant finite-scale operator on $\mathcal{H}_K$.

Finite samples

Let T_K be the sampled operator on $\mathcal{H}_K$, using n uniform samples from $[0, 2\pi)$.
 For every $\varepsilon > 0$,
 $\mathbb{P}(\|T_K - \Lambda_K\|_{\max} \geq \varepsilon) \leq 2d^2 \exp(-Cn\varepsilon^2)$.
 Equivalently, with probability at least $1 - \delta$,
 $\|T_K - \Lambda_K\|_{\max} = O\left(\sqrt{\frac{\log(d/\delta)}{n}}\right)$.

Main results: the spectral picture survives finite scale

$$\mathcal{H}_K = \text{span}\{1, \cos \varphi, \sin \varphi, \dots, \cos(K\varphi), \sin(K\varphi)\}, \quad d = 2K + 1.$$

$$\Lambda_K = \text{diag}(\lambda_0, \lambda_1, \lambda_1, \dots, \lambda_K, \lambda_K), \text{ and } T_K \text{ denotes the relevant finite-scale operator on } \mathcal{H}_K.$$

Finite samples

Thm. 5.5

Let T_K be the sampled operator on $\mathcal{H}_K$, using n uniform samples from $[0, 2\pi)$.

For every $\varepsilon > 0$,

$$\mathbb{P}(\|T_K - \Lambda_K\|_{\max} \geq \varepsilon) \leq 2d^2 \exp(-Cn\varepsilon^2).$$

Equivalently, with probability at least $1 - \delta$,

$$\|T_K - \Lambda_K\|_{\max} = O\left(\sqrt{\frac{\log(d/\delta)}{n}}\right).$$

Finite width

Thm. 5.6

Let $T_K^{(m)}$ be the width- m tangent operator on $\mathcal{H}_K$. Then $\mathbb{E}[T_K^{(m)}] = \Lambda_K$, and

for every $\varepsilon > 0$,

2026-06-11

- Fix one low-frequency Fourier subspace.
- On that subspace, the ideal operator is diagonal.
- Finite samples have an exponential tail in n .
- Frozen finite width is unbiased and has an exponential tail in m .
- Detailed Gram and action bounds are in backup.

Main results: the spectral picture survives finite scale

Finite samples Let T_K be the sampled operator on $\mathcal{H}_K$, using n uniform samples from $[0, 2\pi)$. For every $\varepsilon > 0$, $\mathbb{P}(\ T_K - \Lambda_K\ _{\max} \geq \varepsilon) \leq 2d^2 \exp(-Cn\varepsilon^2).$ Equivalently, with probability at least $1 - \delta$, $\ T_K - \Lambda_K\ _{\max} = O\left(\sqrt{\frac{\log(d/\delta)}{n}}\right).$	Finite width Let $T_K^{(m)}$ be the width-m tangent operator on $\mathcal{H}_K$. Then $\mathbb{E}[T_K^{(m)}] = \Lambda_K$, and for every $\varepsilon > 0$,
--	--

Main results: the spectral picture survives finite scale

$$\mathcal{H}_K = \text{span}\{1, \cos \varphi, \sin \varphi, \dots, \cos(K\varphi), \sin(K\varphi)\}, \quad d = 2K + 1.$$

$$\Lambda_K = \text{diag}(\lambda_0, \lambda_1, \lambda_1, \dots, \lambda_K, \lambda_K), \text{ and } T_K \text{ denotes the relevant finite-scale operator on } \mathcal{H}_K.$$

Finite samples

Thm. 5.5

Let T_K be the sampled operator on $\mathcal{H}_K$, using n uniform samples from $[0, 2\pi)$.

For every $\varepsilon > 0$,

$$\mathbb{P}(\|T_K - \Lambda_K\|_{\max} \geq \varepsilon) \leq 2d^2 \exp(-Cn\varepsilon^2).$$

Equivalently, with probability at least $1 - \delta$,

$$\|T_K - \Lambda_K\|_{\max} = O\left(\sqrt{\frac{\log(d/\delta)}{n}}\right).$$

Finite width

Thm. 5.6

Let $T_K^{(m)}$ be the width- m tangent operator on $\mathcal{H}_K$. Then $\mathbb{E}[T_K^{(m)}] = \Lambda_K$, and

for every $\varepsilon > 0$,

$$\mathbb{P}(\|T_K^{(m)} - \Lambda_K\|_{\max} \geq \varepsilon) \leq 2d^2 \exp(-Cm\varepsilon^2).$$

2026-06-11

- Fix one low-frequency Fourier subspace.
- On that subspace, the ideal operator is diagonal.
- Finite samples have an exponential tail in n .
- Frozen finite width is unbiased and has an exponential tail in m .
- Detailed Gram and action bounds are in backup.

Main results: the spectral picture survives finite scale

$\mathcal{H}_K = \text{span}\{1, \cos \varphi, \sin \varphi, \dots, \cos(K\varphi), \sin(K\varphi)\}, \quad d = 2K + 1.$ $\Lambda_K = \text{diag}(\lambda_0, \lambda_1, \lambda_1, \dots, \lambda_K, \lambda_K)$ and T_K denotes the relevant finite-scale operator on $\mathcal{H}_K$.	
Finite samples	Finite width
Let T_K be the sampled operator on $\mathcal{H}_K$, using n uniform samples from $[0, 2\pi)$. For every $\varepsilon > 0$, $\mathbb{P}(\ T_K - \Lambda_K\ _{\max} \geq \varepsilon) \leq 2d^2 \exp(-Cn\varepsilon^2).$ Equivalently, with probability at least $1 - \delta$, $\ T_K - \Lambda_K\ _{\max} = O\left(\sqrt{\frac{\log(d/\delta)}{n}}\right).$	Let $T_K^{(m)}$ be the width- m tangent operator on $\mathcal{H}_K$. Then $\mathbb{E}[T_K^{(m)}] = \Lambda_K$, and for every $\varepsilon > 0$, $\mathbb{P}(\ T_K^{(m)} - \Lambda_K\ _{\max} \geq \varepsilon) \leq 2d^2 \exp(-Cm\varepsilon^2).$

Main results: the spectral picture survives finite scale

$$\mathcal{H}_K = \text{span}\{1, \cos \varphi, \sin \varphi, \dots, \cos(K\varphi), \sin(K\varphi)\}, \quad d = 2K + 1.$$

$$\Lambda_K = \text{diag}(\lambda_0, \lambda_1, \lambda_1, \dots, \lambda_K, \lambda_K), \text{ and } T_K \text{ denotes the relevant finite-scale operator on } \mathcal{H}_K.$$

Finite samples

Thm. 5.5

Let T_K be the sampled operator on $\mathcal{H}_K$, using n uniform samples from $[0, 2\pi)$.

For every $\varepsilon > 0$,

$$\mathbb{P}(\|T_K - \Lambda_K\|_{\max} \geq \varepsilon) \leq 2d^2 \exp(-Cn\varepsilon^2).$$

Equivalently, with probability at least $1 - \delta$,

$$\|T_K - \Lambda_K\|_{\max} = O\left(\sqrt{\frac{\log(d/\delta)}{n}}\right).$$

Finite width

Thm. 5.6

Let $T_K^{(m)}$ be the width- m tangent operator on $\mathcal{H}_K$. Then $\mathbb{E}[T_K^{(m)}] = \Lambda_K$, and

for every $\varepsilon > 0$,

$$\mathbb{P}(\|T_K^{(m)} - \Lambda_K\|_{\max} \geq \varepsilon) \leq 2d^2 \exp(-Cm\varepsilon^2).$$

Equivalently, with probability at least $1 - \delta$,

2026-06-11

- Fix one low-frequency Fourier subspace.
- On that subspace, the ideal operator is diagonal.
- Finite samples have an exponential tail in n .
- Frozen finite width is unbiased and has an exponential tail in m .
- Detailed Gram and action bounds are in backup.

Main results: the spectral picture survives finite scale

$$\mathcal{H}_K = \text{span}\{1, \cos \varphi, \sin \varphi, \dots, \cos(K\varphi), \sin(K\varphi)\}, \quad d = 2K + 1.$$

$$\Lambda_K = \text{diag}(\lambda_0, \lambda_1, \lambda_1, \dots, \lambda_K, \lambda_K) \text{ and } T_K \text{ denotes the relevant finite-scale operator on } \mathcal{H}_K.$$

Finite samples Thm. 5.5

Let T_K be the sampled operator on $\mathcal{H}_K$, using n uniform samples from $[0, 2\pi)$. For every $\varepsilon > 0$,

$$\mathbb{P}(\|T_K - \Lambda_K\|_{\max} \geq \varepsilon) \leq 2d^2 \exp(-Cn\varepsilon^2).$$

Equivalently, with probability at least $1 - \delta$,

$$\|T_K - \Lambda_K\|_{\max} = O\left(\sqrt{\frac{\log(d/\delta)}{n}}\right).$$

Finite width Thm. 5.6

Let $T_K^{(m)}$ be the width- m tangent operator on $\mathcal{H}_K$. Then $\mathbb{E}[T_K^{(m)}] = \Lambda_K$, and for every $\varepsilon > 0$,

$$\mathbb{P}(\|T_K^{(m)} - \Lambda_K\|_{\max} \geq \varepsilon) \leq 2d^2 \exp(-Cm\varepsilon^2).$$

Equivalently, with probability at least $1 - \delta$,

Main results: the spectral picture survives finite scale

$$\mathcal{H}_K = \text{span}\{1, \cos \varphi, \sin \varphi, \dots, \cos(K\varphi), \sin(K\varphi)\}, \quad d = 2K + 1.$$

$$\Lambda_K = \text{diag}(\lambda_0, \lambda_1, \lambda_1, \dots, \lambda_K, \lambda_K), \text{ and } T_K \text{ denotes the relevant finite-scale operator on } \mathcal{H}_K.$$

Finite samples

Thm. 5.5

Let T_K be the sampled operator on $\mathcal{H}_K$, using n uniform samples from $[0, 2\pi)$.

For every $\varepsilon > 0$,

$$\mathbb{P}(\|T_K - \Lambda_K\|_{\max} \geq \varepsilon) \leq 2d^2 \exp(-Cn\varepsilon^2).$$

Equivalently, with probability at least $1 - \delta$,

$$\|T_K - \Lambda_K\|_{\max} = O\left(\sqrt{\frac{\log(d/\delta)}{n}}\right).$$

Finite width

Thm. 5.6

Let $T_K^{(m)}$ be the width- m tangent operator on $\mathcal{H}_K$. Then $\mathbb{E}[T_K^{(m)}] = \Lambda_K$, and

for every $\varepsilon > 0$,

$$\mathbb{P}(\|T_K^{(m)} - \Lambda_K\|_{\max} \geq \varepsilon) \leq 2d^2 \exp(-Cm\varepsilon^2).$$

Equivalently, with probability at least $1 - \delta$,

$$\|T_K^{(m)} - \Lambda_K\|_{\max} = O\left(\sqrt{\frac{\log(d/\delta)}{m}}\right).$$

2026-06-11

- Fix one low-frequency Fourier subspace.
- On that subspace, the ideal operator is diagonal.
- Finite samples have an exponential tail in n .
- Frozen finite width is unbiased and has an exponential tail in m .
- Detailed Gram and action bounds are in backup.

Main results: the spectral picture survives finite scale

$$\mathcal{H}_K = \text{span}\{1, \cos \varphi, \sin \varphi, \dots, \cos(K\varphi), \sin(K\varphi)\}, \quad d = 2K + 1.$$

$$\Lambda_K = \text{diag}(\lambda_0, \lambda_1, \lambda_1, \dots, \lambda_K, \lambda_K) \text{ and } T_K \text{ denotes the relevant finite-scale operator on } \mathcal{H}_K.$$

Finite samples

Thm. 5.5

Let T_K be the sampled operator on $\mathcal{H}_K$, using n uniform samples from $[0, 2\pi)$.

For every $\varepsilon > 0$,

$$\mathbb{P}(\|T_K - \Lambda_K\|_{\max} \geq \varepsilon) \leq 2d^2 \exp(-Cn\varepsilon^2).$$

Equivalently, with probability at least $1 - \delta$,

$$\|T_K - \Lambda_K\|_{\max} = O\left(\sqrt{\frac{\log(d/\delta)}{n}}\right).$$

Finite width

Thm. 5.6

Let $T_K^{(m)}$ be the width- m tangent operator on $\mathcal{H}_K$. Then $\mathbb{E}[T_K^{(m)}] = \Lambda_K$, and

for every $\varepsilon > 0$,

$$\mathbb{P}(\|T_K^{(m)} - \Lambda_K\|_{\max} \geq \varepsilon) \leq 2d^2 \exp(-Cm\varepsilon^2).$$

Equivalently, with probability at least $1 - \delta$,

$$\|T_K^{(m)} - \Lambda_K\|_{\max} = O\left(\sqrt{\frac{\log(d/\delta)}{m}}\right).$$

Main results: the spectral picture survives finite scale

$$\mathcal{H}_K = \text{span}\{1, \cos \varphi, \sin \varphi, \dots, \cos(K\varphi), \sin(K\varphi)\}, \quad d = 2K + 1.$$

$$\Lambda_K = \text{diag}(\lambda_0, \lambda_1, \lambda_1, \dots, \lambda_K, \lambda_K), \text{ and } T_K \text{ denotes the relevant finite-scale operator on } \mathcal{H}_K.$$

Finite samples

Thm. 5.5

Let T_K be the sampled operator on $\mathcal{H}_K$, using n uniform samples from $[0, 2\pi)$.

For every $\varepsilon > 0$,

$$\mathbb{P}(\|T_K - \Lambda_K\|_{\max} \geq \varepsilon) \leq 2d^2 \exp(-Cn\varepsilon^2).$$

Equivalently, with probability at least $1 - \delta$,

$$\|T_K - \Lambda_K\|_{\max} = O\left(\sqrt{\frac{\log(d/\delta)}{n}}\right).$$

Finite width

Thm. 5.6

Let $T_K^{(m)}$ be the width- m tangent operator on $\mathcal{H}_K$. Then $\mathbb{E}[T_K^{(m)}] = \Lambda_K$, and

for every $\varepsilon > 0$,

$$\mathbb{P}(\|T_K^{(m)} - \Lambda_K\|_{\max} \geq \varepsilon) \leq 2d^2 \exp(-Cm\varepsilon^2).$$

Equivalently, with probability at least $1 - \delta$,

$$\|T_K^{(m)} - \Lambda_K\|_{\max} = O\left(\sqrt{\frac{\log(d/\delta)}{m}}\right).$$

What this gives us: a way to predict how the learning dynamics change with n and m . In particular, the finite-sample block tells us how to precondition the operator and reduce the frequency-wise gap in learning rates.

2026-06-11

- Fix one low-frequency Fourier subspace.
- On that subspace, the ideal operator is diagonal.
- Finite samples have an exponential tail in n .
- Frozen finite width is unbiased and has an exponential tail in m .
- Detailed Gram and action bounds are in backup.

Main results: the spectral picture survives finite scale

Finite samples Let T_K be the sampled operator on $\mathcal{H}_K$, using n uniform samples from $[0, 2\pi)$. For every $\varepsilon > 0$, $\mathbb{P}(\ T_K - \Lambda_K\ _{\max} \geq \varepsilon) \leq 2d^2 \exp(-Cn\varepsilon^2).$ Equivalently, with probability at least $1 - \delta$, $\ T_K - \Lambda_K\ _{\max} = O\left(\sqrt{\frac{\log(d/\delta)}{n}}\right).$	Finite width Let $T_K^{(m)}$ be the width-m tangent operator on $\mathcal{H}_K$. Then $\mathbb{E}[T_K^{(m)}] = \Lambda_K$, and for every $\varepsilon > 0$, $\mathbb{P}(\ T_K^{(m)} - \Lambda_K\ _{\max} \geq \varepsilon) \leq 2d^2 \exp(-Cm\varepsilon^2).$ Equivalently, with probability at least $1 - \delta$, $\ T_K^{(m)} - \Lambda_K\ _{\max} = O\left(\sqrt{\frac{\log(d/\delta)}{m}}\right).$
--	--

What this gives us: a way to predict how the learning dynamics change with n and m . In particular, the finite-sample block tells us how to precondition the operator and reduce the frequency-wise gap in learning rates.

A finite-width effect not explained by the NTK at initialization

After controlling the operator at initialization, the next question is whether training changes the operator enough to matter.

2026-06-11

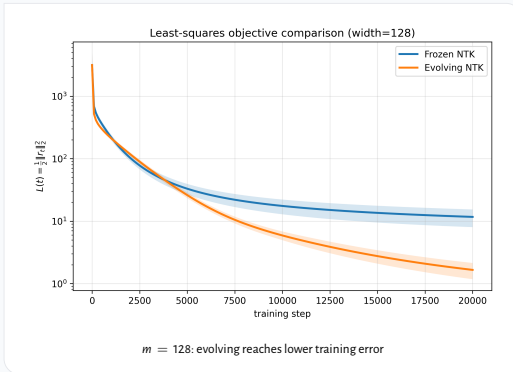
- So far, the story has been: start from the ideal operator, then control finite-sample and frozen finite-width errors.
- But this still leaves out normal training.
- In normal training, the tangent kernel is allowed to move:

$$T_0^{(m)} \rightsquigarrow T_t^{(m)}.$$

- The question is whether that movement actually matters.
- At $m = 128$ and $m = 256$, it clearly does: the evolving kernel reaches lower training error than the frozen one.
- At $m = 1024$, the reversal appears: the frozen kernel reaches the lower final loss.
- So kernel evolution seems to be a genuine small-width effect.
- This is the main tension: finite width can improve learning by changing the operator, but it also breaks the clean spectral reference model.

A finite-width effect not explained by the NTK at initialization

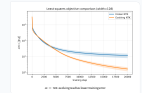
After controlling the operator at initialization, the next question is whether training changes the operator enough to matter.



2026-06-11

A finite-width effect not explained by the NTK at initialization

After controlling the operator at initialization, the next question is whether training changes the operator enough to matter.



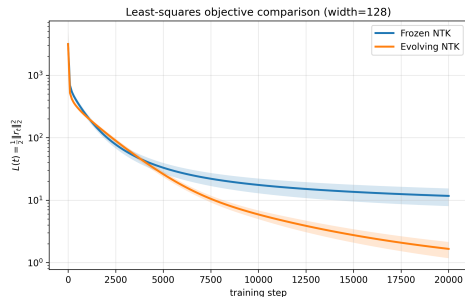
- So far, the story has been: start from the ideal operator, then control finite-sample and frozen finite-width errors.
- But this still leaves out normal training.
- In normal training, the tangent kernel is allowed to move:

$$T_0^{(m)} \rightsquigarrow T_t^{(m)}.$$

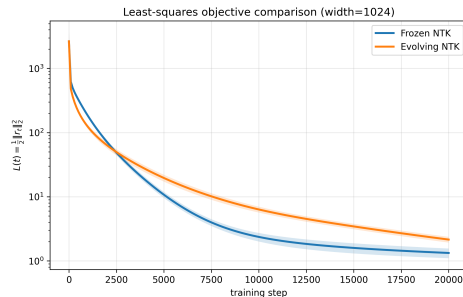
- The question is whether that movement actually matters.
- At $m = 128$ and $m = 256$, it clearly does: the evolving kernel reaches lower training error than the frozen one.
- At $m = 1024$, the reversal appears: the frozen kernel reaches the lower final loss.
- So kernel evolution seems to be a genuine small-width effect.
- This is the main tension: finite width can improve learning by changing the operator, but it also breaks the clean spectral reference model.

A finite-width effect not explained by the NTK at initialization

After controlling the operator at initialization, the next question is whether training changes the operator enough to matter.



$m = 128$: evolving reaches lower training error

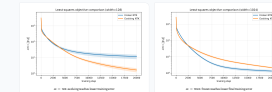


$m = 1024$: frozen reaches lower final training error

2026-06-11

A finite-width effect not explained by the NTK at initialization

After controlling the operator at initialization, the next question is whether training changes the operator enough to matter.



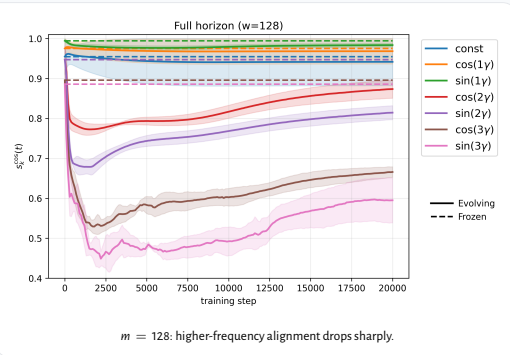
- So far, the story has been: start from the ideal operator, then control finite-sample and frozen finite-width errors.
- But this still leaves out normal training.
- In normal training, the tangent kernel is allowed to move:

$$T_0^{(m)} \rightsquigarrow T_t^{(m)}.$$

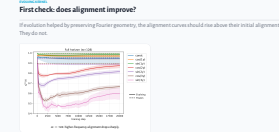
- The question is whether that movement actually matters.
- At $m = 128$ and $m = 256$, it clearly does: the evolving kernel reaches lower training error than the frozen one.
- At $m = 1024$, the reversal appears: the frozen kernel reaches the lower final loss.
- So kernel evolution seems to be a genuine small-width effect.
- This is the main tension: finite width can improve learning by changing the operator, but it also breaks the clean spectral reference model.

First check: does alignment improve?

If evolution helped by preserving Fourier geometry, the alignment curves should rise above their initial alignment. They do not.



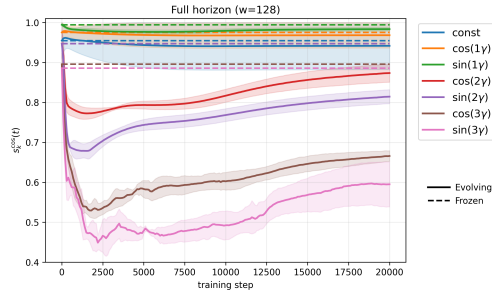
2026-06-11



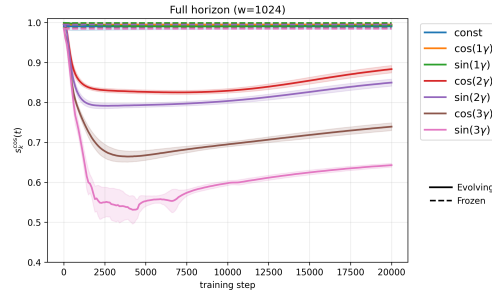
- First possible explanation is geometric: maybe training helps because the evolving kernel aligns its eigenspaces better with the Fourier modes.
- These plots rule out that simple story.
- Width 128: low modes stay aligned, but higher-frequency blocks lose alignment.
- Width 1024: frozen init already very well aligned; evolving again moves several higher-frequency blocks away from the Fourier reference.
- So the small-width improvement is not from better Fourier alignment.

First check: does alignment improve?

If evolution helped by preserving Fourier geometry, the alignment curves should rise above their initial alignment. They do not.



$m = 128$: higher-frequency alignment drops sharply.



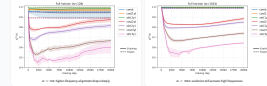
$m = 1024$: evolution still worsens high frequencies.

2026-06-11

- First possible explanation is geometric: maybe training helps because the evolving kernel aligns its eigenspaces better with the Fourier modes.
- These plots rule out that simple story.
- Width 128: low modes stay aligned, but higher-frequency blocks lose alignment.
- Width 1024: frozen init already very well aligned; evolving again moves several higher-frequency blocks away from the Fourier reference.
- So the small-width improvement is not from better Fourier alignment.

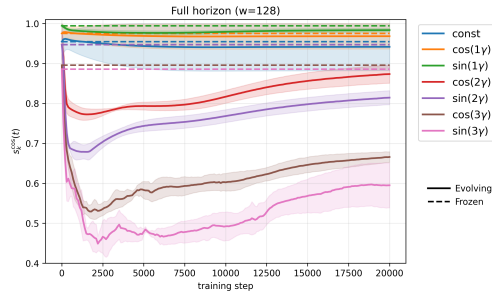
First check: does alignment improve?

If evolution helped by preserving Fourier geometry, the alignment curves should rise above their initial alignment. They do not.

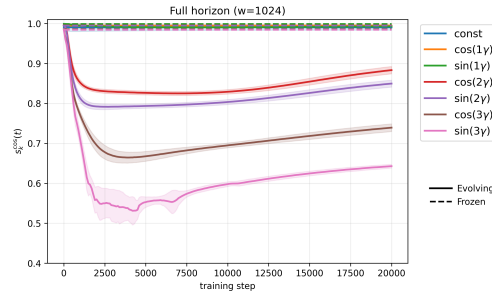


First check: does alignment improve?

If evolution helped by preserving Fourier geometry, the alignment curves should rise above their initial alignment. They do not.



$m = 128$: higher-frequency alignment drops sharply.

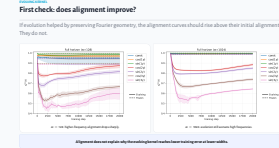


$m = 1024$: evolution still worsens high frequencies.

Alignment does not explain why the evolving kernel reaches lower training error at lower widths.

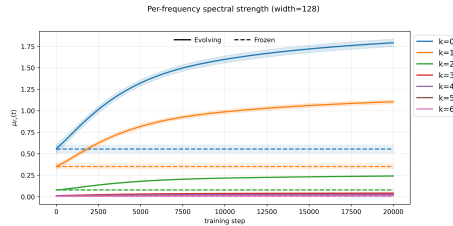
2026-06-11

- First possible explanation is geometric: maybe training helps because the evolving kernel aligns its eigenspaces better with the Fourier modes.
- These plots rule out that simple story.
- Width 128: low modes stay aligned, but higher-frequency blocks lose alignment.
- Width 1024: frozen init already very well aligned; evolving again moves several higher-frequency blocks away from the Fourier reference.
- So the small-width improvement is not from better Fourier alignment.



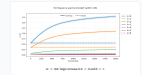
What changes: low-frequency strength rises

The moving kernel becomes stronger on some low-frequency blocks, even without better Fourier alignment.



$m = 128$: large increase in $k = 0$ and $k = 1$.

The moving kernel becomes stronger on some low-frequency blocks, even without better Fourier alignment.

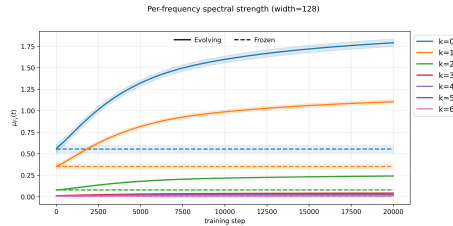


2026-06-11

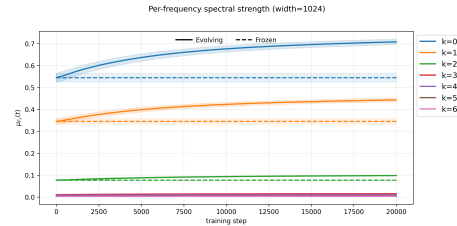
- Second explanation: strength. Even without better Fourier alignment, the operator can become stronger on some low-frequency blocks.
- That's what the spectral-strength plots suggest.
- Width 128: constant block and first frequency rise sharply; smaller rise for $k = 2$.
- Width 1024: same pattern, much weaker.
- Doesn't prove a mechanism, but suggests the low-frequency preference is still involved in the evolving-kernel behaviour.

What changes: low-frequency strength rises

The moving kernel becomes stronger on some low-frequency blocks, even without better Fourier alignment.



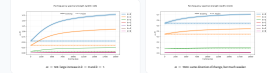
$m = 128$: large increase in $k = 0$ and $k = 1$.



$m = 1024$: same direction of change, but much weaker.

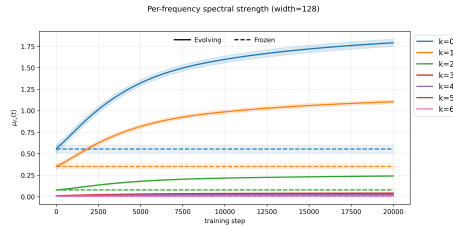
2026-06-11

- Second explanation: strength. Even without better Fourier alignment, the operator can become stronger on some low-frequency blocks.
- That's what the spectral-strength plots suggest.
- Width 128: constant block and first frequency rise sharply; smaller rise for $k = 2$.
- Width 1024: same pattern, much weaker.
- Doesn't prove a mechanism, but suggests the low-frequency preference is still involved in the evolving-kernel behaviour.

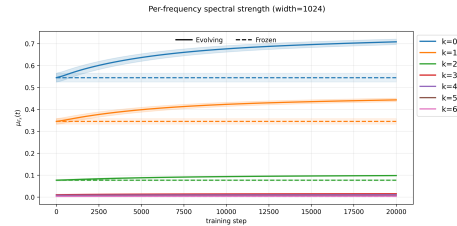


What changes: low-frequency strength rises

The moving kernel becomes stronger on some low-frequency blocks, even without better Fourier alignment.



$m = 128$: large increase in $k = 0$ and $k = 1$.



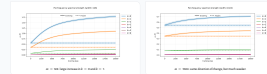
$m = 1024$: same direction of change, but much weaker.

The low-frequency preference is still visible, but the mechanism is not fully explained.

2026-06-11

What changes: low-frequency strength rises

The moving kernel becomes stronger on some low-frequency blocks, even without better Fourier alignment.

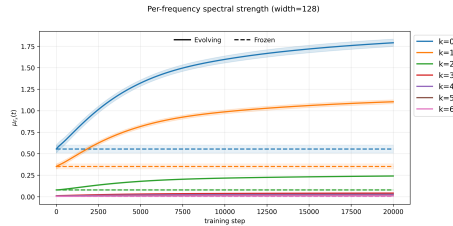


The low-frequency preference is still visible, but the mechanism is not fully explained.

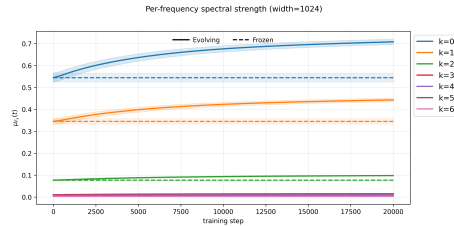
- Second explanation: strength. Even without better Fourier alignment, the operator can become stronger on some low-frequency blocks.
- That's what the spectral-strength plots suggest.
- Width 128: constant block and first frequency rise sharply; smaller rise for $k = 2$.
- Width 1024: same pattern, much weaker.
- Doesn't prove a mechanism, but suggests the low-frequency preference is still involved in the evolving-kernel behaviour.

What changes: low-frequency strength rises

The moving kernel becomes stronger on some low-frequency blocks, even without better Fourier alignment.



$m = 128$: large increase in $k = 0$ and $k = 1$.

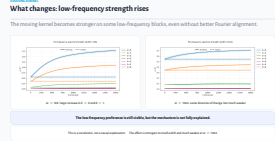


$m = 1024$: same direction of change, but much weaker.

The low-frequency preference is still visible, but the mechanism is not fully explained.

This is a correlation, not a causal explanation. The effect is strongest at small width and much weaker at $m = 1024$.

2026-06-11



- Second explanation: strength. Even without better Fourier alignment, the operator can become stronger on some low-frequency blocks.
- That's what the spectral-strength plots suggest.
- Width 128: constant block and first frequency rise sharply; smaller rise for $k = 2$.
- Width 1024: same pattern, much weaker.
- Doesn't prove a mechanism, but suggests the low-frequency preference is still involved in the evolving-kernel behaviour.

CONCLUSION

What we can now say

The three questions from the start, answered.

1 What sets the learning speed of each frequency?

- ▶ Mode-wise rates are eigenvalues of the NTK; $\lambda_k \sim k^{-2}$, so low frequencies are learned first.

2026-06-11

Map each answer back to the three questions posed early in the talk.

CONCLUSION

What we can now say

The three questions from the start, answered.

1 What sets the learning speed of each frequency?

- ▶ Mode-wise rates are eigenvalues of the NTK; $\lambda_k \sim k^{-2}$, so low frequencies are learned first.

CONCLUSION

What we can now say

The three questions from the start, answered.

1 What sets the learning speed of each frequency?

- ▶ Mode-wise rates are eigenvalues of the NTK; $\lambda_k \sim k^{-2}$, so low frequencies are learned first.

2 Does this survive with only n sampled points? RQ1

- ▶ Yes. Finite samples preserve the picture with explicit $O(n^{-1/2})$ control.

2026-06-11

Map each answer back to the three questions posed early in the talk.

What we can now say

The three questions from the start, answered.

1 What sets the learning speed of each frequency?

- ▶ Mode-wise rates are eigenvalues of the NTK; $\lambda_k \sim k^{-2}$, so low frequencies are learned first.

2 Does this survive with only n sampled points? RQ1

- ▶ Yes. Finite samples preserve the picture with explicit $O(n^{-1/2})$ control.

CONCLUSION

What we can now say

The three questions from the start, answered.

1 What sets the learning speed of each frequency?

- ▶ Mode-wise rates are eigenvalues of the NTK; $\lambda_k \sim k^{-2}$, so low frequencies are learned first.

2 Does this survive with only n sampled points? RQ1

- ▶ Yes. Finite samples preserve the picture with explicit $O(n^{-1/2})$ control.

3 Does it survive with finitely many neurons, at init and during training? RQ2

- ▶ At initialisation, yes, with explicit $O(m^{-1/2})$ control. Once the kernel evolves the clean spectral picture no longer fully describes training: we observe increased low-frequency spectral strength alongside lower training error, but the cause is still open.

2026-06-11

Map each answer back to the three questions posed early in the talk.

conclusion

What we can now say

The three questions from the start, answered.

1

What sets the learning speed of each frequency?

▶ Mode-wise rates are eigenvalues of the NTK; $\lambda_k \sim k^{-2}$, so low frequencies are learned first.

2

Does this survive with only n sampled points? RQ1

▶ Yes. Finite samples preserve the picture with explicit $O(n^{-1/2})$ control.

3

Does it survive with finitely many neurons, at init and during training? RQ2

▶ At initialisation, yes, with explicit $O(m^{-1/2})$ control. Once the kernel evolves the clean spectral picture no longer fully describes training: we observe increased low-frequency spectral strength alongside lower training error, but the cause is still open.

Which assumptions carry the result?

The exact Fourier picture is local to the clean setting; the operator viewpoint is what survives.

Clean setting: uniform data on S^1 + 1-hidden-layer + ReLU + frozen kernel + fixed $\mathcal{H}_K$
 $\implies$ Fourier modes are exact directions, with explicit rates λ_k .

2026-06-11

- Main message: the exact result depends on a very specific clean setting.
- Uniform S^1 : this gives convolution, hence Fourier modes are exact eigenfunctions.
- Non-uniform density on the circle: not a convolution anymore; the basis becomes the eigenfunctions of the density-weighted operator.
- Gaussian inputs: different domain and measure; expect Hermite-polynomial-type directions, not Fourier modes.
- One hidden layer: this is what makes the kernel and eigenvalues explicitly computable in this thesis.
- Deeper isotropic networks can still have a rotation-invariant infinite-width kernel on uniform S^1 , so Fourier modes may remain the eigenbasis, but the eigenvalues change and are usually given recursively rather than by this closed form.
- For other architectures, such as attention blocks or convolutional models, the relevant operator is architecture-dependent; the preferred directions need not be the same Fourier modes.
- ReLU: gives a computable spectrum with polynomial decay, $\lambda_k = O(k^{-2})$ here.
- Smooth activation does not mean high frequencies are learned faster.
- It means the kernel eigenvalues decay faster as frequency increases.
- Since the learning rate equals the eigenvalue, high-frequency modes become even slower relative to low frequencies.
- So smoother activations can sharpen the low-frequency bias.
- Frozen kernel: this is where the finite-width theory applies.
- Evolving kernel: initial spectrum no longer fully explains training; our results are empirical and suggest increased low-frequency strength at small width.

Which assumptions carry the result?

The exact Fourier picture is local to the clean setting; the operator viewpoint is what survives.

Clean setting: uniform data on S^1 + 1-hidden-layer + ReLU + frozen kernel + fixed $\mathcal{H}_K$
 $\implies$ Fourier modes are exact directions, with explicit rates λ_k .

- **Change the data distribution:** Fourier modes need not be the right basis anymore. On a non-uniform circle, learning follows the eigenfunctions of a density-weighted operator; with Gaussian inputs, the analogue is closer to Hermite-polynomial directions.

2026-06-11

- Main message: the exact result depends on a very specific clean setting.
- Uniform S^1 : this gives convolution, hence Fourier modes are exact eigenfunctions.
- Non-uniform density on the circle: not a convolution anymore; the basis becomes the eigenfunctions of the density-weighted operator.
- Gaussian inputs: different domain and measure; expect Hermite-polynomial-type directions, not Fourier modes.
- One hidden layer: this is what makes the kernel and eigenvalues explicitly computable in this thesis.
- Deeper isotropic networks can still have a rotation-invariant infinite-width kernel on uniform S^1 , so Fourier modes may remain the eigenbasis, but the eigenvalues change and are usually given recursively rather than by this closed form.
- For other architectures, such as attention blocks or convolutional models, the relevant operator is architecture-dependent; the preferred directions need not be the same Fourier modes.
- ReLU: gives a computable spectrum with polynomial decay, $\lambda_k = O(k^{-2})$ here.
- Smooth activation does not mean high frequencies are learned faster.
- It means the kernel eigenvalues decay faster as frequency increases.
- Since the learning rate equals the eigenvalue, high-frequency modes become even slower relative to low frequencies.
- So smoother activations can sharpen the low-frequency bias.
- Frozen kernel: this is where the finite-width theory applies.
- Evolving kernel: initial spectrum no longer fully explains training; our results are empirical and suggest increased low-frequency strength at small width.

The exact Fourier picture is local to the clean setting; the operator viewpoint is what survives.

Clean setting: uniform data on S^1 + 1-hidden-layer + ReLU + frozen kernel + fixed $\mathcal{H}_K$
 $\implies$ Fourier modes are exact directions, with explicit rates λ_k .

- **Change the data distribution:** Fourier modes need not be the right basis anymore. On a non-uniform circle, learning follows the eigenfunctions of a density-weighted operator; with Gaussian inputs, the analogue is closer to Hermite-polynomial directions.

Which assumptions carry the result?

The exact Fourier picture is local to the clean setting; the operator viewpoint is what survives.

Clean setting: uniform data on S^1 + 1-hidden-layer + ReLU + frozen kernel + fixed $\mathcal{H}_K$
 $\implies$ Fourier modes are exact directions, with explicit rates λ_k .

- **Change the data distribution:** Fourier modes need not be the right basis anymore. On a non-uniform circle, learning follows the eigenfunctions of a density-weighted operator; with Gaussian inputs, the analogue is closer to Hermite-polynomial directions.
- **Change the architecture:** deeper or different networks change the kernel and hence the spectrum. Symmetry may still give Fourier-like directions, but the rates are different and less explicit.

2026-06-11

- Main message: the exact result depends on a very specific clean setting.
- Uniform S^1 : this gives convolution, hence Fourier modes are exact eigenfunctions.
- Non-uniform density on the circle: not a convolution anymore; the basis becomes the eigenfunctions of the density-weighted operator.
- Gaussian inputs: different domain and measure; expect Hermite-polynomial-type directions, not Fourier modes.
- One hidden layer: this is what makes the kernel and eigenvalues explicitly computable in this thesis.
- Deeper isotropic networks can still have a rotation-invariant infinite-width kernel on uniform S^1 , so Fourier modes may remain the eigenbasis, but the eigenvalues change and are usually given recursively rather than by this closed form.
- For other architectures, such as attention blocks or convolutional models, the relevant operator is architecture-dependent; the preferred directions need not be the same Fourier modes.
- ReLU: gives a computable spectrum with polynomial decay, $\lambda_k = O(k^{-2})$ here.
- Smooth activation does not mean high frequencies are learned faster.
- It means the kernel eigenvalues decay faster as frequency increases.
- Since the learning rate equals the eigenvalue, high-frequency modes become even slower relative to low frequencies.
- So smoother activations can sharpen the low-frequency bias.
- Frozen kernel: this is where the finite-width theory applies.
- Evolving kernel: initial spectrum no longer fully explains training; our results are empirical and suggest increased low-frequency strength at small width.

WHICH ASSUMPTIONS CARRY THE RESULT?
 The exact Fourier picture is local to the clean setting; the operator viewpoint is what survives.

Clean setting: uniform data on S^1 + 1-hidden-layer + ReLU + frozen kernel + fixed $\mathcal{H}_K$
 $\implies$ Fourier modes are exact directions, with explicit rates λ_k .

- **Change the data distribution:** Fourier modes need not be the right basis anymore. On a non-uniform circle, learning follows the eigenfunctions of a density-weighted operator; with Gaussian inputs, the analogue is closer to Hermite-polynomial directions.
- **Change the architecture:** deeper or different networks change the kernel and hence the spectrum. Symmetry may still give Fourier-like directions, but the rates are different and less explicit.

Which assumptions carry the result?

The exact Fourier picture is local to the clean setting; the operator viewpoint is what survives.

Clean setting: uniform data on S^1 + 1-hidden-layer + ReLU + frozen kernel + fixed $\mathcal{H}_K$

$\implies$ Fourier modes are exact directions, with explicit rates λ_k .

- **Change the data distribution:** Fourier modes need not be the right basis anymore. On a non-uniform circle, learning follows the eigenfunctions of a density-weighted operator; with Gaussian inputs, the analogue is closer to Hermite-polynomial directions.
- **Change the architecture:** deeper or different networks change the kernel and hence the spectrum. Symmetry may still give Fourier-like directions, but the rates are different and less explicit.
- **Change the activation:** the operator may keep the same eigenbasis under the same symmetry, but the eigenvalues change. Smoother activations usually make eigenvalues decay faster with frequency, so fine detail becomes even slower.

2026-06-11

- Main message: the exact result depends on a very specific clean setting.
- Uniform S^1 : this gives convolution, hence Fourier modes are exact eigenfunctions.
- Non-uniform density on the circle: not a convolution anymore; the basis becomes the eigenfunctions of the density-weighted operator.
- Gaussian inputs: different domain and measure; expect Hermite-polynomial-type directions, not Fourier modes.
- One hidden layer: this is what makes the kernel and eigenvalues explicitly computable in this thesis.
- Deeper isotropic networks can still have a rotation-invariant infinite-width kernel on uniform S^1 , so Fourier modes may remain the eigenbasis, but the eigenvalues change and are usually given recursively rather than by this closed form.
- For other architectures, such as attention blocks or convolutional models, the relevant operator is architecture-dependent; the preferred directions need not be the same Fourier modes.
- ReLU: gives a computable spectrum with polynomial decay, $\lambda_k = O(k^{-2})$ here.
- Smooth activation does not mean high frequencies are learned faster.
- It means the kernel eigenvalues decay faster as frequency increases.
- Since the learning rate equals the eigenvalue, high-frequency modes become even slower relative to low frequencies.
- So smoother activations can sharpen the low-frequency bias.
- Frozen kernel: this is where the finite-width theory applies.
- Evolving kernel: initial spectrum no longer fully explains training; our results are empirical and suggest increased low-frequency strength at small width.

Which assumptions carry the result?

The exact Fourier picture is local to the clean setting; the operator viewpoint is what survives.

Clean setting: uniform data on S^1 + 1-hidden-layer + ReLU + frozen kernel + fixed $\mathcal{H}_K$

$\implies$ Fourier modes are exact directions, with explicit rates λ_k .

- **Change the data distribution:** Fourier modes need not be the right basis anymore. On a non-uniform circle, learning follows the eigenfunctions of a density-weighted operator; with Gaussian inputs, the analogue is closer to Hermite-polynomial directions.
- **Change the architecture:** deeper or different networks change the kernel and hence the spectrum. Symmetry may still give Fourier-like directions, but the rates are different and less explicit.
- **Change the activation:** the operator may keep the same eigenbasis under the same symmetry, but the eigenvalues change. Smoother activations usually make eigenvalues decay faster with frequency, so fine detail becomes even slower.

Which assumptions carry the result?

The exact Fourier picture is local to the clean setting; the operator viewpoint is what survives.

Clean setting: uniform data on S^1 + 1-hidden-layer + ReLU + frozen kernel + fixed $\mathcal{H}_K$
 $\implies$ Fourier modes are exact directions, with explicit rates λ_k .

- **Change the data distribution:** Fourier modes need not be the right basis anymore. On a non-uniform circle, learning follows the eigenfunctions of a density-weighted operator; with Gaussian inputs, the analogue is closer to Hermite-polynomial directions.
- **Change the architecture:** deeper or different networks change the kernel and hence the spectrum. Symmetry may still give Fourier-like directions, but the rates are different and less explicit.
- **Change the activation:** the operator may keep the same eigenbasis under the same symmetry, but the eigenvalues change. Smoother activations usually make eigenvalues decay faster with frequency, so fine detail becomes even slower.
- **Let the kernel evolve:** the initial spectrum is no longer a complete description. In our experiments, small-width training increases low-frequency strength, but this is an empirical diagnostic, not a causal theory.

2026-06-11

- Main message: the exact result depends on a very specific clean setting.
- Uniform S^1 : this gives convolution, hence Fourier modes are exact eigenfunctions.
- Non-uniform density on the circle: not a convolution anymore; the basis becomes the eigenfunctions of the density-weighted operator.
- Gaussian inputs: different domain and measure; expect Hermite-polynomial-type directions, not Fourier modes.
- One hidden layer: this is what makes the kernel and eigenvalues explicitly computable in this thesis.
- Deeper isotropic networks can still have a rotation-invariant infinite-width kernel on uniform S^1 , so Fourier modes may remain the eigenbasis, but the eigenvalues change and are usually given recursively rather than by this closed form.
- For other architectures, such as attention blocks or convolutional models, the relevant operator is architecture-dependent; the preferred directions need not be the same Fourier modes.
- ReLU: gives a computable spectrum with polynomial decay, $\lambda_k = O(k^{-2})$ here.
- Smooth activation does not mean high frequencies are learned faster.
- It means the kernel eigenvalues decay faster as frequency increases.
- Since the learning rate equals the eigenvalue, high-frequency modes become even slower relative to low frequencies.
- So smoother activations can sharpen the low-frequency bias.
- Frozen kernel: this is where the finite-width theory applies.
- Evolving kernel: initial spectrum no longer fully explains training; our results are empirical and suggest increased low-frequency strength at small width.

Which assumptions carry the result?

The exact Fourier picture is local to the clean setting; the operator viewpoint is what survives.

Clean setting: uniform data on S^1 + 1-hidden-layer + ReLU + frozen kernel + fixed $\mathcal{H}_K$
 $\implies$ Fourier modes are exact directions, with explicit rates λ_k .

- **Change the data distribution:** Fourier modes need not be the right basis anymore. On a non-uniform circle, learning follows the eigenfunctions of a density-weighted operator; with Gaussian inputs, the analogue is closer to Hermite-polynomial directions.
- **Change the architecture:** deeper or different networks change the kernel and hence the spectrum. Symmetry may still give Fourier-like directions, but the rates are different and less explicit.
- **Change the activation:** the operator may keep the same eigenbasis under the same symmetry, but the eigenvalues change. Smoother activations usually make eigenvalues decay faster with frequency, so fine detail becomes even slower.
- **Let the kernel evolve:** the initial spectrum is no longer a complete description. In our experiments, small-width training increases low-frequency strength, but this is an empirical diagnostic, not a causal theory.

Which assumptions carry the result?

The exact Fourier picture is local to the clean setting; the operator viewpoint is what survives.

Clean setting: uniform data on S^1 + 1-hidden-layer + ReLU + frozen kernel + fixed $\mathcal{H}_K$

$\implies$ Fourier modes are exact directions, with explicit rates λ_k .

- **Change the data distribution:** Fourier modes need not be the right basis anymore. On a non-uniform circle, learning follows the eigenfunctions of a density-weighted operator; with Gaussian inputs, the analogue is closer to Hermite-polynomial directions.
- **Change the architecture:** deeper or different networks change the kernel and hence the spectrum. Symmetry may still give Fourier-like directions, but the rates are different and less explicit.
- **Change the activation:** the operator may keep the same eigenbasis under the same symmetry, but the eigenvalues change. Smoother activations usually make eigenvalues decay faster with frequency, so fine detail becomes even slower.
- **Let the kernel evolve:** the initial spectrum is no longer a complete description. In our experiments, small-width training increases low-frequency strength, but this is an empirical diagnostic, not a causal theory.
- **Leave the fixed block $\mathcal{H}_K$:** the bounds do not control all frequencies at once. High frequencies are harder because their eigenvalues are smaller, closer together, and more sensitive to perturbations.

2026-06-11

- Main message: the exact result depends on a very specific clean setting.
- Uniform S^1 : this gives convolution, hence Fourier modes are exact eigenfunctions.
- Non-uniform density on the circle: not a convolution anymore; the basis becomes the eigenfunctions of the density-weighted operator.
- Gaussian inputs: different domain and measure; expect Hermite-polynomial-type directions, not Fourier modes.
- One hidden layer: this is what makes the kernel and eigenvalues explicitly computable in this thesis.
- Deeper isotropic networks can still have a rotation-invariant infinite-width kernel on uniform S^1 , so Fourier modes may remain the eigenbasis, but the eigenvalues change and are usually given recursively rather than by this closed form.
- For other architectures, such as attention blocks or convolutional models, the relevant operator is architecture-dependent; the preferred directions need not be the same Fourier modes.
- ReLU: gives a computable spectrum with polynomial decay, $\lambda_k = O(k^{-2})$ here.
- Smooth activation does not mean high frequencies are learned faster.
- It means the kernel eigenvalues decay faster as frequency increases.
- Since the learning rate equals the eigenvalue, high-frequency modes become even slower relative to low frequencies.
- So smoother activations can sharpen the low-frequency bias.
- Frozen kernel: this is where the finite-width theory applies.
- Evolving kernel: initial spectrum no longer fully explains training; our results are empirical and suggest increased low-frequency strength at small width.

Which assumptions carry the result?

The exact Fourier picture is local to the clean setting; the operator viewpoint is what survives.

Clean setting: uniform data on S^1 + 1-hidden-layer + ReLU + frozen kernel + fixed $\mathcal{H}_K$

$\implies$ Fourier modes are exact directions, with explicit rates λ_k .

- **Change the data distribution:** Fourier modes need not be the right basis anymore. On a non-uniform circle, learning follows the eigenfunctions of a density-weighted operator; with Gaussian inputs, the analogue is closer to Hermite-polynomial directions.
- **Change the architecture:** deeper or different networks change the kernel and hence the spectrum. Symmetry may still give Fourier-like directions, but the rates are different and less explicit.
- **Change the activation:** the operator may keep the same eigenbasis under the same symmetry, but the eigenvalues change. Smoother activations usually make eigenvalues decay faster with frequency, so fine detail becomes even slower.
- **Let the kernel evolve:** the initial spectrum is no longer a complete description. In our experiments, small-width training increases low-frequency strength, but this is an empirical diagnostic, not a causal theory.
- **Leave the fixed block $\mathcal{H}_K$:** the bounds do not control all frequencies at once. High frequencies are harder because their eigenvalues are smaller, closer together, and more sensitive to perturbations.

Which assumptions carry the result?

The exact Fourier picture is local to the clean setting; the operator viewpoint is what survives.

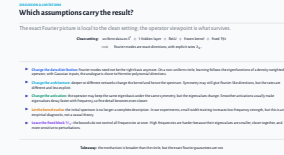
Clean setting: uniform data on S^1 + 1-hidden-layer + ReLU + frozen kernel + fixed $\mathcal{H}_K$

$\implies$ Fourier modes are exact directions, with explicit rates λ_k .

- **Change the data distribution:** Fourier modes need not be the right basis anymore. On a non-uniform circle, learning follows the eigenfunctions of a density-weighted operator; with Gaussian inputs, the analogue is closer to Hermite-polynomial directions.
- **Change the architecture:** deeper or different networks change the kernel and hence the spectrum. Symmetry may still give Fourier-like directions, but the rates are different and less explicit.
- **Change the activation:** the operator may keep the same eigenbasis under the same symmetry, but the eigenvalues change. Smoother activations usually make eigenvalues decay faster with frequency, so fine detail becomes even slower.
- **Let the kernel evolve:** the initial spectrum is no longer a complete description. In our experiments, small-width training increases low-frequency strength, but this is an empirical diagnostic, not a causal theory.
- **Leave the fixed block $\mathcal{H}_K$:** the bounds do not control all frequencies at once. High frequencies are harder because their eigenvalues are smaller, closer together, and more sensitive to perturbations.

Takeaway: the mechanism is broader than the circle, but the exact Fourier guarantees are not.

2026-06-11



- Main message: the exact result depends on a very specific clean setting.
- Uniform S^1 : this gives convolution, hence Fourier modes are exact eigenfunctions.
- Non-uniform density on the circle: not a convolution anymore; the basis becomes the eigenfunctions of the density-weighted operator.
- Gaussian inputs: different domain and measure; expect Hermite-polynomial-type directions, not Fourier modes.
- One hidden layer: this is what makes the kernel and eigenvalues explicitly computable in this thesis.
- Deeper isotropic networks can still have a rotation-invariant infinite-width kernel on uniform S^1 , so Fourier modes may remain the eigenbasis, but the eigenvalues change and are usually given recursively rather than by this closed form.
- For other architectures, such as attention blocks or convolutional models, the relevant operator is architecture-dependent; the preferred directions need not be the same Fourier modes.
- ReLU: gives a computable spectrum with polynomial decay, $\lambda_k = O(k^{-2})$ here.
- Smooth activation does not mean high frequencies are learned faster.
- It means the kernel eigenvalues decay faster as frequency increases.
- Since the learning rate equals the eigenvalue, high-frequency modes become even slower relative to low frequencies.
- So smoother activations can sharpen the low-frequency bias.
- Frozen kernel: this is where the finite-width theory applies.
- Evolving kernel: initial spectrum no longer fully explains training; our results are empirical and suggest increased low-frequency strength at small width.

Where this sits, and where it connects

► A concrete version of an ideal theory

- NTK theory already gives the mechanism: in the lazy limit, training is governed by a fixed operator, and its spectrum sets the learning rates. This thesis turns that into a fully explicit reference model on S^1 , for the shallow ReLU network actually studied here (Jacot et al., 2018; Cao et al., 2020).

2026-06-11

- Purpose: position the thesis, not recap the whole talk.
- Case study: known NTK mechanism, but fully explicit for this model.
- Bowman: closest comparison; they are broader, we are more explicit.
- Finite scale: split sampling and width instead of bundling them.
- Lazy boundary: fixed-kernel theory is the reference, not the whole story.
- Practice: PINNs and coordinate networks already fight spectral imbalance.
- Fourier features: encode coordinates with sine/cosine features.
- SIREN: uses sine activations for fine-detail coordinate signals.
- Boundary: connects through spectrum, not direct application claims.

Where this sits, and where it connects

► A concrete version of an ideal theory

- NTK theory already gives the mechanism: in the lazy limit, training is governed by a fixed operator, and its spectrum sets the learning rates. This thesis turns that into a fully explicit reference model on S^1 , for the shallow ReLU network actually studied here (Jacot et al., 2018; Cao et al., 2020).

► Less general than Bowman–Montúfar, but more inspectable

- Bowman & Montúfar compare finite networks trained on finite samples to ideal kernel dynamics in a broader setting. Here the setting is much smaller, but the perturbations are separated: sampling error and frozen-width error can be measured against known Fourier eigenvalues, mode by mode (Bowman & Montúfar, 2022).

2026-06-11

- Purpose: position the thesis, not recap the whole talk.
- Case study: known NTK mechanism, but fully explicit for this model.
- Bowman: closest comparison; they are broader, we are more explicit.
- Finite scale: split sampling and width instead of bundling them.
- Lazy boundary: fixed-kernel theory is the reference, not the whole story.
- Practice: PINNs and coordinate networks already fight spectral imbalance.
- Fourier features: encode coordinates with sine/cosine features.
- SIREN: uses sine activations for fine-detail coordinate signals.
- Boundary: connects through spectrum, not direct application claims.

- A concrete version of an ideal theory
 - NTK theory already gives the mechanism: in the lazy limit, training is governed by a fixed operator and its spectrum sets the learning rates. This thesis turns that into a fully explicit reference model on S^1 , for the shallow ReLU network actually studied here (Jacot et al., 2018; Cao et al., 2020).
- Less general than Bowman–Montúfar, but more inspectable
 - Bowman & Montúfar compare finite networks trained on finite samples to ideal kernel dynamics in a broader setting. Here the setting is much smaller, but the perturbations are separated: sampling error and frozen-width error can be measured against known Fourier eigenvalues, mode by mode (Bowman & Montúfar, 2022).

Where this sits, and where it connects

► A concrete version of an ideal theory

- NTK theory already gives the mechanism: in the lazy limit, training is governed by a fixed operator, and its spectrum sets the learning rates. This thesis turns that into a fully explicit reference model on S^1 , for the shallow ReLU network actually studied here (Jacot et al., 2018; Cao et al., 2020).

► Less general than Bowman–Montúfar, but more inspectable

- Bowman & Montúfar compare finite networks trained on finite samples to ideal kernel dynamics in a broader setting. Here the setting is much smaller, but the perturbations are separated: sampling error and frozen-width error can be measured against known Fourier eigenvalues, mode by mode (Bowman & Montúfar, 2022).

► A boundary case for the lazy-training picture

- Lazy-training theory explains why the fixed kernel can be a useful approximation, but also why it is not feature learning. The evolving-kernel experiments here are a small diagnostic of that boundary: once the kernel moves, the initial spectrum no longer tells the full story (Chizat et al., 2019).

2026-06-11

- Purpose: position the thesis, not recap the whole talk.
- Case study: known NTK mechanism, but fully explicit for this model.
- Bowman: closest comparison; they are broader, we are more explicit.
- Finite scale: split sampling and width instead of bundling them.
- Lazy boundary: fixed-kernel theory is the reference, not the whole story.
- Practice: PINNs and coordinate networks already fight spectral imbalance.
- Fourier features: encode coordinates with sine/cosine features.
- SIREN: uses sine activations for fine-detail coordinate signals.
- Boundary: connects through spectrum, not direct application claims.

- **A concrete version of an ideal theory**
 - NTK theory already gives the mechanism: in the lazy limit, training is governed by a fixed operator and its spectrum sets the learning rates. This thesis turns that into a fully explicit reference model on S^1 , for the shallow ReLU network actually studied here (Jacot et al., 2018; Cao et al., 2020).
- **Less general than Bowman–Montúfar, but more inspectable**
 - Bowman & Montúfar compare finite networks trained on finite samples to ideal kernel dynamics in a broader setting. Here the setting is much smaller, but the perturbations are separated: sampling error and frozen-width error can be measured against known Fourier eigenvalues, mode by mode (Bowman & Montúfar, 2022).
- **A boundary case for the lazy-training picture**
 - Lazy training theory explains why the fixed kernel can be a useful approximation, but also why it is not feature learning. The evolving kernel experiments here are a small diagnostic of that boundary: once the kernel moves, the initial spectrum no longer tells the full story (Chizat et al., 2019).

Where this sits, and where it connects

► A concrete version of an ideal theory

- NTK theory already gives the mechanism: in the lazy limit, training is governed by a fixed operator, and its spectrum sets the learning rates. This thesis turns that into a fully explicit reference model on S^1 , for the shallow ReLU network actually studied here (Jacot et al., 2018; Cao et al., 2020).

► Less general than Bowman–Montúfar, but more inspectable

- Bowman & Montúfar compare finite networks trained on finite samples to ideal kernel dynamics in a broader setting. Here the setting is much smaller, but the perturbations are separated: sampling error and frozen-width error can be measured against known Fourier eigenvalues, mode by mode (Bowman & Montúfar, 2022).

► A boundary case for the lazy-training picture

- Lazy-training theory explains why the fixed kernel can be a useful approximation, but also why it is not feature learning. The evolving-kernel experiments here are a small diagnostic of that boundary: once the kernel moves, the initial spectrum no longer tells the full story (Chizat et al., 2019).

► A clean model of a practical spectral problem

- In PINNs, different solution components can converge at very different speeds; one explanation is imbalance in the kernel eigenvalues, and some methods try to rescale those directions (Wang et al., 2022).
- In coordinate networks, such as position $\mapsto$ colour, shape, or audio value, fine detail is often hard to learn. Fourier features add sinusoidal coordinate encodings, and SIREN uses sinusoidal activations; both change the effective spectrum so high-frequency detail is less suppressed (Tancik et al., 2020; Sitzmann et al., 2020).

2026-06-11

- Purpose: position the thesis, not recap the whole talk.
- Case study: known NTK mechanism, but fully explicit for this model.
- Bowman: closest comparison; they are broader, we are more explicit.
- Finite scale: split sampling and width instead of bundling them.
- Lazy boundary: fixed-kernel theory is the reference, not the whole story.
- Practice: PINNs and coordinate networks already fight spectral imbalance.
- Fourier features: encode coordinates with sine/cosine features.
- SIREN: uses sine activations for fine-detail coordinate signals.
- Boundary: connects through spectrum, not direct application claims.

Where this sits, and where it connects
► A concrete version of an ideal theory - NTK theory already gives the mechanism: in the lazy limit, training is governed by a fixed operator, and its spectrum sets the learning rates. This thesis turns that into a fully explicit reference model on S^1, for the shallow ReLU network actually studied here (Jacot et al., 2018; Cao et al., 2020). ► Less general than Bowman–Montúfar, but more inspectable - Bowman & Montúfar compare finite networks trained on finite samples to ideal kernel dynamics in a broader setting. Here the setting is much smaller, but the perturbations are separated: sampling error and frozen-width error can be measured against known Fourier eigenvalues, mode by mode (Bowman & Montúfar, 2022). ► A boundary case for the lazy-training picture - Lazy-training theory explains why the fixed kernel can be a useful approximation, but also why it is not feature learning. The evolving-kernel experiments here are a small diagnostic of that boundary: once the kernel moves, the initial spectrum no longer tells the full story (Chizat et al., 2019). ► A clean model of a practical spectral problem - In PINNs, different solution components can converge at very different speeds; one explanation is imbalance in the kernel eigenvalues, and some methods try to rescale those directions (Wang et al., 2022). - In coordinate networks, such as position $\mapsto$ colour, shape, or audio value, fine detail is often hard to learn. Fourier features add sinusoidal coordinate encodings, and SIREN uses sinusoidal activations; both change the effective spectrum so high-frequency detail is less suppressed (Tancik et al., 2020; Sitzmann et al., 2020).

Where this sits, and where it connects

► A concrete version of an ideal theory

- NTK theory already gives the mechanism: in the lazy limit, training is governed by a fixed operator, and its spectrum sets the learning rates. This thesis turns that into a fully explicit reference model on S^1 , for the shallow ReLU network actually studied here (Jacot et al., 2018; Cao et al., 2020).

► Less general than Bowman–Montúfar, but more inspectable

- Bowman & Montúfar compare finite networks trained on finite samples to ideal kernel dynamics in a broader setting. Here the setting is much smaller, but the perturbations are separated: sampling error and frozen-width error can be measured against known Fourier eigenvalues, mode by mode (Bowman & Montúfar, 2022).

► A boundary case for the lazy-training picture

- Lazy-training theory explains why the fixed kernel can be a useful approximation, but also why it is not feature learning. The evolving-kernel experiments here are a small diagnostic of that boundary: once the kernel moves, the initial spectrum no longer tells the full story (Chizat et al., 2019).

► A clean model of a practical spectral problem

- In PINNs, different solution components can converge at very different speeds; one explanation is imbalance in the kernel eigenvalues, and some methods try to rescale those directions (Wang et al., 2022).
- In coordinate networks, such as position $\mapsto$ colour, shape, or audio value, fine detail is often hard to learn. Fourier features add sinusoidal coordinate encodings, and SIREN uses sinusoidal activations; both change the effective spectrum so high-frequency detail is less suppressed (Tancik et al., 2020; Sitzmann et al., 2020).

Still, a result on the uniform circle is far from a guarantee about a deployed model.

2026-06-11

- Purpose: position the thesis, not recap the whole talk.
- Case study: known NTK mechanism, but fully explicit for this model.
- Bowman: closest comparison; they are broader, we are more explicit.
- Finite scale: split sampling and width instead of bundling them.
- Lazy boundary: fixed-kernel theory is the reference, not the whole story.
- Practice: PINNs and coordinate networks already fight spectral imbalance.
- Fourier features: encode coordinates with sine/cosine features.
- SIREN: uses sine activations for fine-detail coordinate signals.
- Boundary: connects through spectrum, not direct application claims.

► A concrete version of an ideal theory	15
► Less general than Bowman–Montúfar, but more inspectable	16
► A boundary case for the lazy-training picture	17
► A clean model of a practical spectral problem	18

THANK YOU

Questions?

Thank you for listening.

2026-06-11

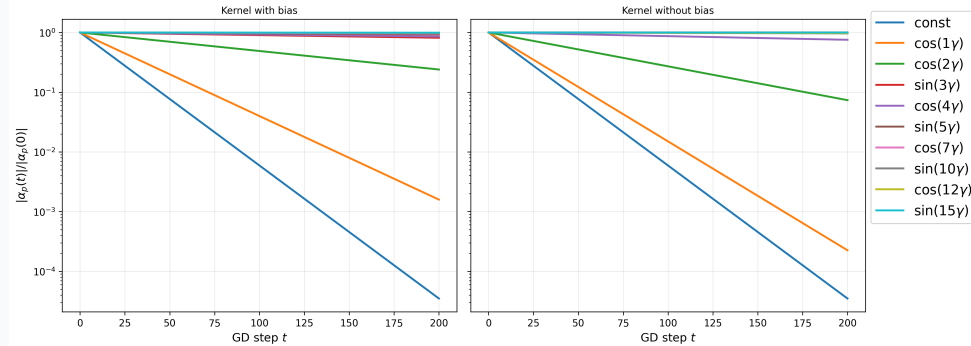
Thank you for listening. I am happy to take questions.

Questions?

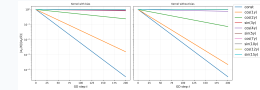
Thank you for listening.

Backup

Continuum prediction: spectral bias



Continuum prediction: spectral bias



2026-06-11

NTK: from parameter updates to an operator on functions

Gradient flow on the weights induces a linear flow on the residual, governed by an operator built from the network's own gradients.

Setup

Least-squares loss on $L^2(\mu)$, with residual $r_t = f_t - y$:

$$\mathcal{L}(\theta) = \frac{1}{2} \|f_m(\cdot; \theta) - y\|_{L^2(\mu)}^2.$$

The empirical NTK is the gradient inner product

$$\Theta_t^{(m)}(\varphi, \psi) = \langle \nabla_{\theta} f_t(\varphi), \nabla_{\theta} f_t(\psi) \rangle,$$

with operator $(T_t^{(m)} f)(\varphi) = \int \Theta_t^{(m)}(\varphi, \psi) f(\psi) d\mu(\psi)$.

Residual flow

Gradient flow on θ and the chain rule give a linear flow on the residual:

$$\partial_t r_t = -T_t^{(m)} r_t, \quad \frac{d}{dt} \frac{1}{2} \|r_t\|^2 = -\langle r_t, T_t^{(m)} r_t \rangle.$$

Two regimes inside one equation:

- **Frozen**: replace $T_t^{(m)}$ by $T_0^{(m)}$ (kernel regression on the initial features).
- **Evolving**: keep the full time dependence of $T_t^{(m)}$.

As $m \rightarrow \infty, T_0^{(m)} \rightarrow T$, the deterministic continuum operator.

2026-06-11

NTK: from parameter updates to an operator on functions

Gradient flow on the weights induces a linear flow on the residual, governed by an operator built from the network's own gradients.

Setup	Residual flow
Least-squares loss on $L^2(\mu)$ with residual $r_t = f_t - y$: $\mathcal{L}(\theta) = \frac{1}{2} \ f_m(\cdot; \theta) - y\ _{L^2(\mu)}^2$	Gradient flow on θ and the chain rule give a linear flow on the residual: $\partial_t r_t = -T_t^{(m)} r_t, \quad \frac{d}{dt} \frac{1}{2} \ r_t\ ^2 = -\langle r_t, T_t^{(m)} r_t \rangle$
The empirical NTK is the gradient inner product: $\Theta_t^{(m)}(\varphi, \psi) = \langle \nabla_{\theta} f_t(\varphi), \nabla_{\theta} f_t(\psi) \rangle$, with operator $(T_t^{(m)} f)(\varphi) = \int \Theta_t^{(m)}(\varphi, \psi) f(\psi) d\mu(\psi)$.	The eigenvalues define two equations: ► Frozen : replace $T_t^{(m)}$ by $T_0^{(m)}$ (kernel regression on the initial features). ► Evolving : keep the full time dependence of $T_t^{(m)}$. As $m \rightarrow \infty, T_0^{(m)} \rightarrow T$, the deterministic continuum operator.

Finite-scale bounds: precise statements

$\mathcal{H}_K$: span of the Fourier modes up to frequency $K, d = 2K + 1$; Φ : those modes evaluated at the n samples; $\|\cdot\|_{\max}$: entrywise max. Main-slide notation: $T_K^{(n)}$ corresponds to H , and $T_K^{(m)}$ corresponds to $B_m^{(K)}$.

Finite samples

Thm. 5.5

Sampled kernel, Gram, and compressed operator:

$$A_{ij} = \frac{1}{n} T(\theta_i, \theta_j), \quad G = \frac{1}{n} \Phi^\top \Phi, \quad H = \frac{1}{n} \Phi^\top A \Phi$$

With probability at least $1 - \delta$:

$$\|G - I\|_{\max} \leq C_1 \sqrt{\frac{\log(d/\delta)}{n}} \quad (\text{orthogonality})$$

$$\|H - \Lambda^{(n)}\|_{\max} \leq C_2 \sqrt{\frac{\log(d/\delta)}{n}} \quad (\text{eigenvalues})$$

$$\|A\Phi - \Phi\Lambda^{(n)}\|_{\max} \leq C_3 \sqrt{\frac{\log(nd/\delta)}{n}} \quad (\text{kernel action})$$

$\Lambda^{(n)}$: ideal eigenvalues with an $O(1/n)$ correction; $\mathbb{E}[H] = \Lambda^{(n)}$ (Prop. 5.4).

Frozen finite width

Thm. 5.6

Compress the width- m tangent operator at initialization to $\mathcal{H}_K$:

$$B_m^{(K)} = P_K T_0^{(m)} P_K, \quad \Lambda_K = P_K T P_K$$

Unbiased at every width, and concentrating:

$$\mathbb{E} B_m^{(K)} = \Lambda_K$$

$$\|B_m^{(K)} - \Lambda_K\|_{\max} \leq C_4 \sqrt{\frac{\log(d/\delta)}{m}}$$

with probability at least $1 - \delta$; sub-exponential regime omitted.

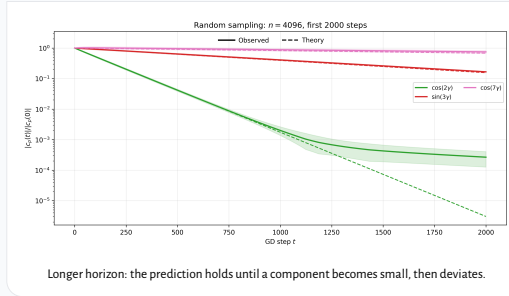
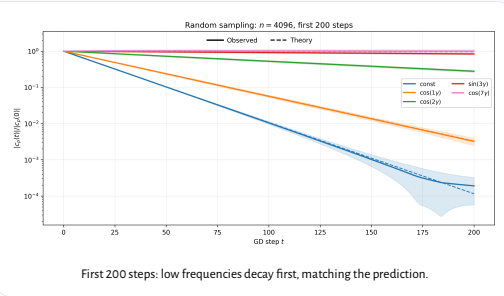
Finite-scale bounds: precise statements

$\mathcal{H}_K$: span of the Fourier modes up to frequency $K, d = 2K + 1$; Φ : those modes evaluated at the n samples; $\|\cdot\|_{\max}$: entrywise max. Main-slide notation: $T_K^{(n)}$ corresponds to H , and $T_K^{(m)}$ corresponds to $B_m^{(K)}$.

Finite samples	Thm. 5.5	Frozen finite width	Thm. 5.6
Sampled kernel, Gram, and compressed operator: $A_{ij} = \frac{1}{n} T(\theta_i, \theta_j), \quad G = \frac{1}{n} \Phi^\top \Phi, \quad H = \frac{1}{n} \Phi^\top A \Phi$		Compress the width- m tangent operator at initialization to $\mathcal{H}_K$: $B_m^{(K)} = P_K T_0^{(m)} P_K, \quad \Lambda_K = P_K T P_K$	
With probability at least $1 - \delta$:		Unbiased at every width, and concentrating: $\mathbb{E} B_m^{(K)} = \Lambda_K$	
$\ G - I\ _{\max} \leq C_1 \sqrt{\frac{\log(d/\delta)}{n}}$ (orthogonality)		$\ B_m^{(K)} - \Lambda_K\ _{\max} \leq C_4 \sqrt{\frac{\log(d/\delta)}{m}}$	
$\ H - \Lambda^{(n)}\ _{\max} \leq C_2 \sqrt{\frac{\log(d/\delta)}{n}}$ (eigenvalues)			
$\ A\Phi - \Phi\Lambda^{(n)}\ _{\max} \leq C_3 \sqrt{\frac{\log(nd/\delta)}{n}}$ (kernel action)			
$\Lambda^{(n)}$: ideal eigenvalues with an $O(1/n)$ correction; $\mathbb{E}[H] = \Lambda^{(n)}$ (Prop. 5.4).			

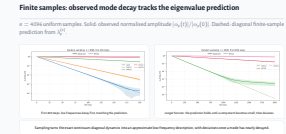
Finite samples: observed mode decay tracks the eigenvalue prediction

$n = 4096$ uniform samples. Solid: observed normalised amplitude $|\alpha_p(t)|/|\alpha_p(0)|$. Dashed: diagonal finite-sample prediction from $\lambda_p^{(n)}$.



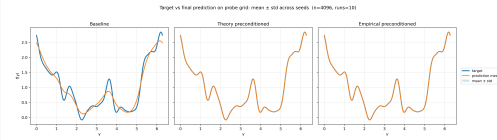
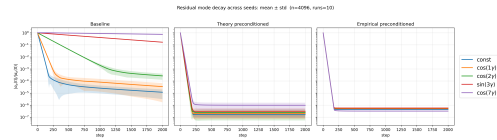
Sampling turns the exact continuum diagonal dynamics into an approximate low-frequency description, with deviations once a mode has nearly decayed.

2026-06-11



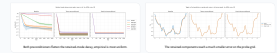
Preconditioning: rescale each retained mode by its eigenvalue

The fixed-kernel dynamics are linear with per-mode rate λ_p . Rescaling the retained Fourier block flattens the rates. Theory: divide by predicted $\lambda_p^{(n)}$. Empirical: invert the observed block $C = G^{-1}H$.



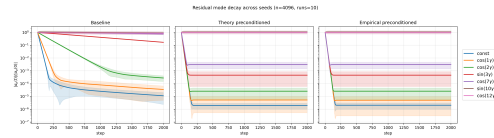
Preconditioning: rescale each retained mode by its eigenvalue

The fixed-kernel dynamics are linear with per-mode rate λ_p . Rescaling the retained Fourier block flattens the rates. Theory: divide by predicted $\lambda_p^{(n)}$. Empirical: invert the observed block $C = G^{-1}H$.

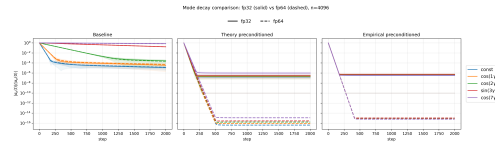


Preconditioning: the two floors that limit it

The preconditioner is built only on the retained block, and is solved in finite precision. Each limit shows up as a different plateau.



Out-of-block modes ($k = 10, 12$) stay near their start; in-block modes plateau higher from coupling.



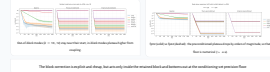
fp32 (solid) vs fp64 (dashed): the preconditioned plateau drops by orders of magnitude, so that floor is numerical ($\sim \kappa u$).

The block correction is explicit and cheap, but acts only inside the retained block and bottoms out at the conditioning-set precision floor.

2026-06-11

Preconditioning: the two floors that limit it

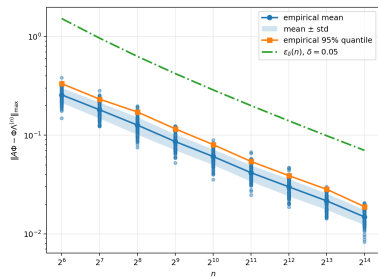
The preconditioner is built only on the retained block, and is solved in finite precision. Each limit shows up as a different plateau.



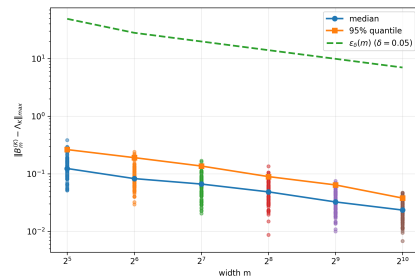
The block correction is explicit and cheap, but acts only inside the retained block and bottoms out at the conditioning-set precision floor.

Concentration bounds: the constants, and where the magnitude bound is loose

2026-06-11

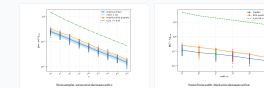


finite samples: action error decreases with n

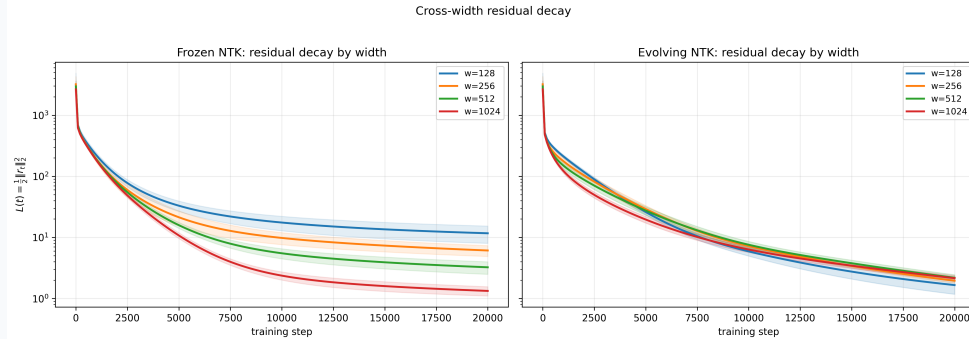


frozen finite width: block error decreases with m

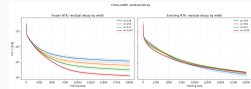
Concentration bounds: the constants, and where the magnitude bound is loose



Frozen versus evolving: cross-width view

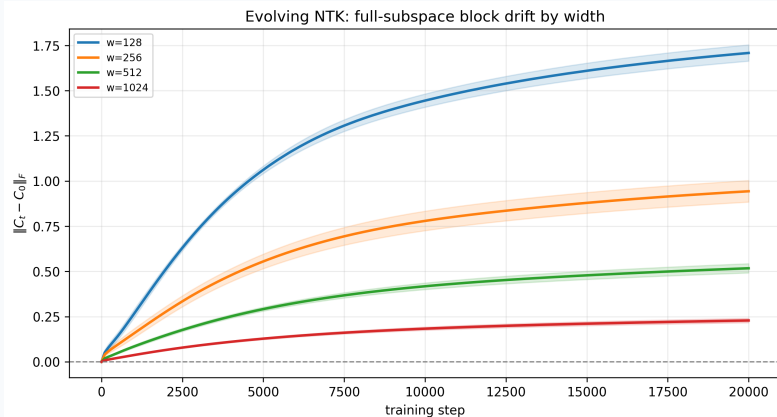


Frozen versus evolving: cross-width view

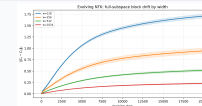


2026-06-11

Kernel movement is a small-width effect

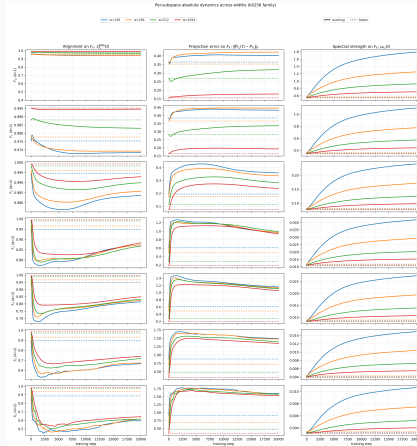


Kernel movement is a small-width effect

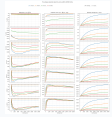


2026-06-11

Full per-subspace evolving-kernel diagnostics



Full per-subspace evolving-kernel diagnostics



2026-06-11

Computing the eigenvalues λ_k

The eigenvalues are the Fourier coefficients of the convolution kernel $\Theta(\delta)$.

From gradients to a closed-form kernel

At init ($a_i, w_i \sim \mathcal{N}(0, I), b_i = 0$), the width- m NTK concentrates on

$$\Theta(x, y) = K_0 + (x^\top y + 1)K_1,$$

with the ReLU expectations (angle $\delta, \cos \delta = x^\top y$)

$$K_1 = \frac{\pi - \delta}{2\pi}, \quad K_0 = \frac{1}{2\pi} (\sin \delta + (\pi - \delta) \cos \delta),$$

so that

$$\Theta(\delta) = \frac{1}{2\pi} (\sin \delta + 2(\pi - \delta) \cos \delta + (\pi - \delta)).$$

Convolution $\Rightarrow$ Fourier diagonal

Θ depends only on $\delta = \varphi - \psi$, so T is a convolution and

$$\lambda_k = \frac{1}{\pi} \int_0^\pi \Theta(\delta) \cos(k\delta) d\delta.$$

The parity integral $\int_0^\pi (\pi - \delta) \cos(m\delta) d\delta = \frac{1 - (-1)^m}{m^2}$ splits even and odd k :

$$\lambda_0 = \frac{1}{4} + \frac{3}{\pi^2}, \quad \lambda_1 = \frac{1}{4} + \frac{1}{\pi^2},$$

$$\lambda_k = \begin{cases} \frac{k^2 + 3}{\pi^2 (k^2 - 1)^2}, & k \geq 2 \text{ even}, \\ \frac{1}{\pi^2 k^2}, & k \geq 3 \text{ odd}. \end{cases}$$

2026-06-11

Computing the eigenvalues λ_k

The eigenvalues are the Fourier coefficients of the convolution kernel $\Theta(\delta)$.

From gradients to a closed-form kernel	Convolution $\Rightarrow$ Fourier diagonal
At init ($a_i, w_i \sim \mathcal{N}(0, I), b_i = 0$), the width- m NTK concentrates on $\Theta(x, y) = K_0 + (x^\top y + 1)K_1$	Θ depends only on $\delta = \varphi - \psi$, so T is a convolution and $\lambda_k = \frac{1}{\pi} \int_0^\pi \Theta(\delta) \cos(k\delta) d\delta$
with the ReLU expectations (angle $\delta, \cos \delta = x^\top y$) $K_1 = \frac{\pi - \delta}{2\pi}, \quad K_0 = \frac{1}{2\pi} (\sin \delta + (\pi - \delta) \cos \delta)$	The parity integral $\int_0^\pi (\pi - \delta) \cos(m\delta) d\delta = \frac{1 - (-1)^m}{m^2}$ splits even and odd k
so that $\Theta(\delta) = \frac{1}{2\pi} (\sin \delta + 2(\pi - \delta) \cos \delta + (\pi - \delta))$	$\lambda_k = \begin{cases} \frac{k^2 + 3}{\pi^2 (k^2 - 1)^2}, & k \geq 2 \text{ even}, \\ \frac{1}{\pi^2 k^2}, & k \geq 3 \text{ odd}. \end{cases}$

The $k = 2$ alignment anomaly

λ_2 follows the generic even- k formula normally. The anomaly is in the geometry: at finite width, $k = 2$ is unusually well aligned with its Fourier subspace.

Observation

- ▶ Projector error grows with frequency, as expected, with one exception: $k = 2$ has the smallest error among the displayed frequencies.
- ▶ It is also the best aligned, sitting above its neighbours rather than on the decreasing trend.

A tentative explanation

- ▶ $k = 0$ gets a direct eigenvalue from the bias; $k = 1$ is tied to the coordinate embedding and is the only odd mode in the no-bias kernel.
- ▶ The finite-width perturbation may therefore act differently on $k = 0, 1$ than on higher blocks, leaving $k = 2$ relatively favoured.

This is an empirical observation, not a theoretical claim; the mechanism is not investigated further.

2026-06-11

The $k = 2$ alignment anomaly

λ_k follows the generic even- k formula normally. The anomaly is in the geometry: at finite width, $k = 2$ is unusually well aligned with its Fourier subspace.

Observation	A tentative explanation
▶ Projector error grows with frequency, as expected, with one exception: $k = 2$ has the smallest error among the displayed frequencies. ▶ It is also the best aligned, sitting above its neighbours rather than on the decreasing trend.	▶ $k = 0$ gets a direct eigenvalue from the bias; $k = 1$ is tied to the coordinate embedding and is the only odd mode in the no-bias kernel. ▶ The finite-width perturbation may therefore act differently on $k = 0, 1$ than on higher blocks, leaving $k = 2$ relatively favoured.

This is an empirical observation, not a theoretical claim; the mechanism is not investigated further.

Eigenvalue ordering: where it holds, and what it depends on

Because the domain and measure are rotation-symmetric, the eigenvalue ordering is also a frequency ordering. The trainable bias is what keeps it strict.

The spectrum is strictly decreasing

$$\begin{array}{ll} \lambda_0 \approx 0.554 & \lambda_1 \approx 0.351 \\ \lambda_2 \approx 0.079 & \lambda_3 \approx 0.011 \\ \lambda_4 \approx 0.0086 & \lambda_5 \approx 0.0041 \end{array}$$

Two interleaved subsequences (even and odd k), both $O(k^{-2})$, with $\lambda_k > \lambda_{k+1}$ throughout.

What the ordering depends on

- ▶ The gap between $k = 0, 1$ and the rest comes from the constant and the coordinate embedding.
- ▶ The odd modes $k \geq 3$ are nonzero *only* because of the trainable hidden bias. Without it, $\lambda_k^{(\text{nb})} = 0$ for odd $k \geq 3$, so those modes are null and never learned by the fixed kernel.
- ▶ Even $k \geq 2$ are unchanged by the bias.

Consistent with bias-free two-layer ReLU models not expressing odd harmonics beyond $k = 1$.

2026-06-11

Eigenvalue ordering: where it holds, and what it depends on

Because the domain and measure are rotation-symmetric, the eigenvalue ordering is also a frequency ordering. The trainable bias is what keeps it strict.

The spectrum is strictly decreasing
 $\lambda_0 \approx 0.554$ $\lambda_1 \approx 0.351$
 $\lambda_2 \approx 0.079$ $\lambda_3 \approx 0.011$
 $\lambda_4 \approx 0.0086$ $\lambda_5 \approx 0.0041$
Two interleaved subsequences (even and odd k), both $O(k^{-2})$, with $\lambda_k > \lambda_{k+1}$ throughout.

What the ordering depends on

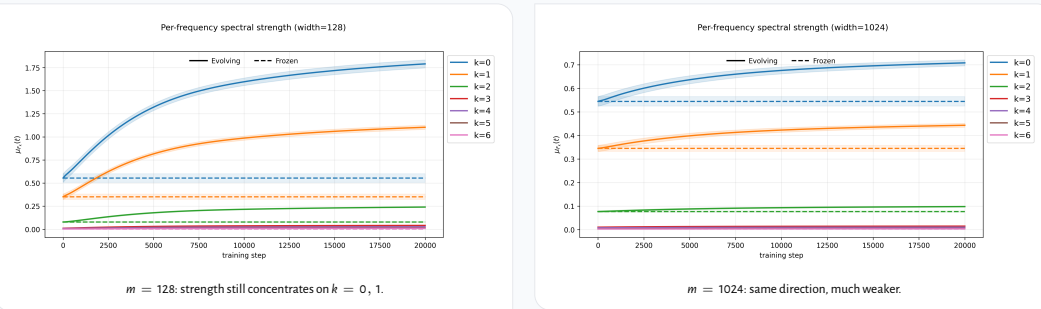
- ▶ The gap between $k = 0, 1$ and the rest comes from the constant and the coordinate embedding.
- ▶ The odd modes $k \geq 3$ are nonzero *only* because of the trainable hidden bias. Without it, $\lambda_k^{(\text{nb})} = 0$ for odd $k \geq 3$, so those modes are null and never learned by the fixed kernel.
- ▶ Even $k \geq 2$ are unchanged by the bias.

Consistent with bias-free two-layer ReLU models not expressing odd harmonics beyond $k = 1$.

Why $k = 0, 1$ gain strength: the parameterisation hypothesis

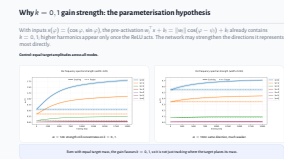
With inputs $x(\varphi) = (\cos \varphi, \sin \varphi)$, the pre-activation $w_j^\top x + b_j = \|w_j\| \cos(\varphi - \psi_j) + b_j$ already contains $k = 0, 1$; higher harmonics appear only once the ReLU acts. The network may strengthen the directions it represents most directly.

Control: equal target amplitudes across all modes.



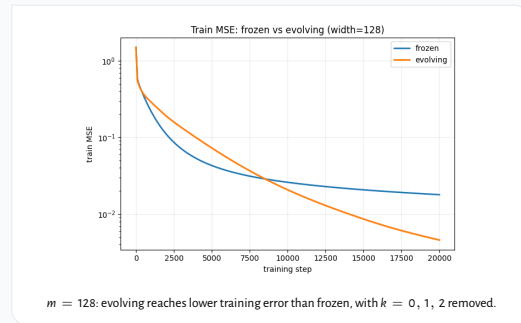
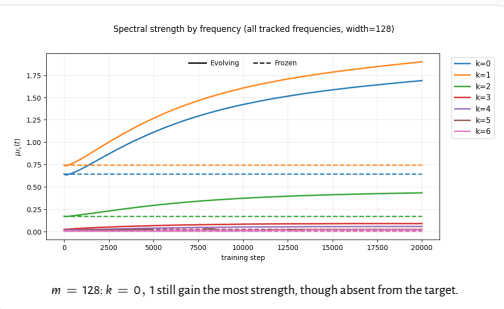
Even with equal target mass, the gain favours $k = 0, 1$, so it is not just tracking where the target places its mass.

2026-06-11



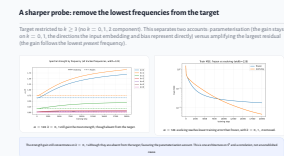
A sharper probe: remove the lowest frequencies from the target

Target restricted to $k \geq 3$ (no $k = 0, 1, 2$ component). This separates two accounts: parameterisation (the gain stays on $k = 0, 1$, the directions the input embedding and bias represent directly) versus amplifying the largest residual (the gain follows the lowest *present* frequency).



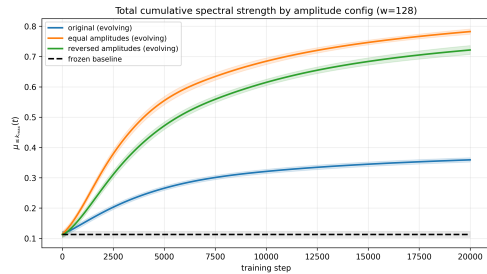
The strength gain still concentrates on $k = 0, 1$ although they are absent from the target, favouring the parameterisation account. This is one architecture on S^1 and a correlation, not an established cause.

2026-06-11

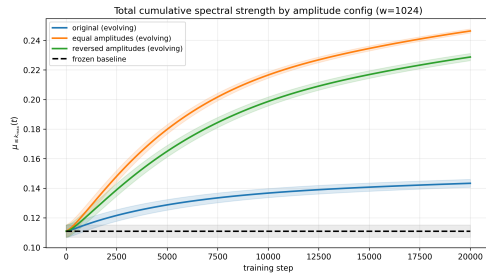


Low-frequency strength gain is robust across amplitude configs

Cumulative low-frequency strength $\mu_{\leq k_{\max}}(t)$ for three amplitude profiles of the target: original, equal, and reversed (largest amplitudes on the highest frequencies).



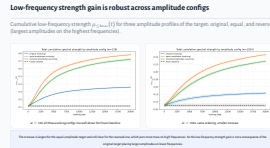
$m = 128$: all three evolving configs rise well above the frozen baseline.



$m = 1024$: same ordering, smaller increase.

The increase is largest for the equal-amplitude target and still clear for the reversed one, which puts more mass on high frequencies. So the low-frequency strength gain is not a consequence of the original target placing larger amplitudes on lower frequencies.

2026-06-11



Mean-field viewpoint: a complementary large-width limit

The NTK is one way to take width to infinity. The mean-field limit is another, and it is what makes the evolving-kernel regime natural.

NTK / lazy limit

- ▶ Track a *fixed* tangent operator at initialization.
- ▶ On S^1 it is diagonal in the Fourier basis, so learning is read off eigenvalues and eigenspaces.
- ▶ Features barely move; this is the regime our bounds control.

Mean-field limit

With scaling $f_\theta = \frac{1}{m} \sum_\alpha v_\alpha \varphi(w_\alpha^\top x)$, the output depends on the neurons only through their empirical measure

$$\mu_m = \frac{1}{m} \sum_\alpha \delta_{\theta_\alpha} \longrightarrow \mu_t.$$

- ▶ Training is a flow of the *distribution* of neurons, not a fixed kernel.
- ▶ This is the language for the feature-learning side where the kernel evolves.

2026-06-11

Mean-field viewpoint: a complementary large-width limit

The NTK is one way to take width to infinity. The mean-field limit is another, and it is what makes the evolving-kernel regime natural.

NTK / lazy limit ▶ Track a fixed tangent operator at initialization. ▶ On S^1 it is diagonal in the Fourier basis, so learning is read off eigenvalues and eigenspaces. ▶ Features barely move; this is the regime our bounds control.	Mean-field limit With scaling $f_\theta = \frac{1}{m} \sum_\alpha v_\alpha \varphi(w_\alpha^\top x)$ the output depends on the neurons only through their empirical measure $\mu_m = \frac{1}{m} \sum_\alpha \delta_{\theta_\alpha} \longrightarrow \mu_t.$ ▶ Training is a flow of the distribution of neurons, not a fixed kernel. ▶ This is the language for the feature-learning side where the kernel evolves.
--	--

References

- ▶ Rahaman, Baratin, Arpit, Draxler, Lin, Hamprecht, Bengio, Courville (2019). *On the Spectral Bias of Neural Networks*. ICML.
- ▶ Jacot, Gabriel, Hongler (2018). *Neural Tangent Kernel: Convergence and Generalization in Neural Networks*. NeurIPS.
- ▶ Bowman, Montúfar (2022). *Spectral Bias Outside the Training Set for Deep Networks in the Kernel Regime*. NeurIPS.
- ▶ Zhang, Bengio, Hardt, Recht, Vinyals (2017). *Understanding Deep Learning Requires Rethinking Generalization*. ICLR.

2026-06-11

- ### References
- ▶ Rahaman, Baratin, Arpit, Draxler, Lin, Hamprecht, Bengio, Courville (2019). *On the Spectral Bias of Neural Networks*. ICML.
 - ▶ Jacot, Gabriel, Hongler (2018). *Neural Tangent Kernel: Convergence and Generalization in Neural Networks*. NeurIPS.
 - ▶ Bowman, Montúfar (2022). *Spectral Bias Outside the Training Set for Deep Networks in the Kernel Regime*. NeurIPS.
 - ▶ Zhang, Bengio, Hardt, Recht, Vinyals (2017). *Understanding Deep Learning Requires Rethinking Generalization*. ICLR.